$F2 \,\|\, WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing the maximum weighted completion time in a two-machine flow shop models a line in which a few high-priority orders pace the shop. The problem is strongly NP-hard, and we give a mixed integer programming formulation for it. We show that an optimal schedule preserves the permutation property and has a block structure: one block per distinct weight, Johnson's order within each block, and the blocks in non-increasing weight order. On this structure we build three methods: a dynamic program that prunes dominated states and runs in $O(nKV^{3K-3})$ time, where K is the number of distinct weights and V the total processing time; a 2-approximation in $O(n \log n)$ time; and a fully polynomial-time approximation scheme that attains a $(1+\varepsilon)$ guarantee for a constant number of weights. For large instances we develop an adapted NEH heuristic and a genetic algorithm. In a study with twenty instances per configuration, the model settles all 440 small instances, and at $K = 2$ the dynamic program certifies the optimum up to eighty jobs on both ranges and to a hundred on the narrower one; what stops it is cost, not correctness. The 2-approximation stays within 3.5 per cent of the optimum on average and within 1.8 per cent of the lower bound, NEH within 2.5 and 4.4, and the genetic algorithm, the weakest on small instances, is the only method whose gap falls as the instance grows, passing both rules beyond about 260 jobs.

Keywords: Flow shop; Dynamic programming; Approximation algorithms;

1 Introduction

The two-machine flow shop is one of the best known models in scheduling. Its structure is that of a plant whose jobs are machined on one machine and finished on another: metal

*Corresponding author. E-mail address: author@email.com

parts are cut and then coated, a casting is rolled after it is poured, a web is printed and then bound. A job must leave the first stage before it can enter the second, and each stage carries one job at a time. Johnson settled the sequencing decision for this model as early as 1954 (Johnson, 1954), and the two-stage structure has remained a standard subject of sequencing theory, with its variants and solution methods reviewed in (Pinedo, 2022).

The criterion of this paper is the maximum weighted completion time, also called the weighted makespan. It charges each job the product of its weight and its completion time and takes the largest of those products: where the makespan asks when the shop falls idle, the weighted makespan asks which job is left waiting and what that wait costs. The weight belongs to the order rather than to the plant as a whole, and in a plant it is rarely a continuum. A contractual penalty for lateness, the capital tied up in work in process and the priority a large customer has bought are three classes rather than a spectrum, and the most costly order is the one carrying the largest of them.

Much the same structure appears outside the factory. A system that serves large language model agents handles every step of a task in two phases: the step is reasoned about in the chain of thought (Wei et al., 2022), and only then acted upon, by calling a tool or running code, in the chain of execution (Yao et al., 2023). A step is a job, the reasoning is the first machine and the acting is the second. A step cannot be acted upon before it has been reasoned about, each phase serves one step at a time, and the time a step spends in a phase is its processing time on that machine. The two phases draw on different resources, the model for the reasoning and the tool, the interpreter or the index for the acting, and the phases of different steps run alongside each other, so that when a step finishes is settled by the phase it is in and by the time the steps ahead of it have kept the resource it waits for. The steps are not equal: one bought by a paid tier, or one whose failure is costly, weighs more than the rest, and that cost is the weight of the job.

Two features of this environment are not a choice of convenience. The two stages stand in series because a step has to be reasoned about before it is acted upon, and merging them would erase the overlap that makes the order of the steps matter at all: reasoning and acting consume different resources, and a step that waits for its tool leaves the model free for the next one. The criterion is a maximum rather than an average for the same reason, since what decides whether the service is acceptable is the step that matters most. On a single machine that step would not be a property of the schedule: its completion time would be settled by its position in the sequence, no earlier stage being there whose delay it could inherit. Two stages in series are what make the completion time of the decisive step depend on the whole schedule.

In scheduling theory the criterion has been studied in several machine environments. Maximum-type criteria have a classical foundation: Lawler's backward rule minimizes, on a single machine, the maximum of arbitrary nondecreasing cost functions in quadratic time (Lawler, 1973), and weighted completion times are read as a maximum rather than a sum

within that framework. In the online setting, where jobs are revealed over time, best-possible algorithms are known for the same criterion on one machine (Li, 2015), and a machine carrying one restricted rate-modifying activity has been treated for it as well (Zuo et al., 2026). On identical parallel machines, randomized approximation schemes have been developed for the weighted makespan (Sun, 2024), and the criterion has been combined with the decision of which orders to accept (Lu et al., 2025). Maximum-type criteria appear in other forms as well, such as the maximum earliness/tardiness cost of a single machine when jobs may be rejected (Guo et al., 2026). In all of these settings, however, a job occupies one machine for its entire processing time, so that the interaction between two consecutive stages, which is what defines the flow shop, does not arise.

The flow shop environment, by contrast, has been studied extensively, but for different criteria. The variant that has attracted most theoretical attention is the proportionate flow shop, in which every machine spends the same time on a given job; it is surveyed, with the ordered flow shop and related variants, in (Panwalkar et al., 2013). Its appeal is that it can often be reduced to a single-machine problem with the same criterion (Panwalkar and Koulamas, 2015), so that results proved for one machine carry over. Weighted objectives have been treated within it as well, but the objective is the sum of the weighted completion times rather than their maximum (Shakhlevich et al., 1998). For the general two-machine flow shop, the makespan is settled by Johnson's rule, and the criteria studied beyond it are the sums of the completion times and of the weighted completion times, which are NP-hard already with two machines (Garey et al., 1976); the most recent study combines the makespan with the decision of which orders to reject (Shabtay and Gerstl, 2024).

The criterion and the machine environment are therefore each well established, while their combination is not: the criterion has been read on one machine at a time, and the flow shop for the makespan and for the sums of completion times, but not for their maximum. The two sets of results do not combine. The reductions that make the proportionate flow shop tractable require the processing times to be proportionate, and the general two-machine flow shop does not satisfy that assumption. More fundamentally, a total weighted completion time decomposes over the jobs and can be accumulated while a schedule is built, which is what allows a dynamic program to compare partial schedules with one another; a maximum does not decompose in this way, since a partial schedule carries no information about its final value until it is complete. Optimizing $F2 \parallel WC_{\max}$ therefore requires structural properties of a kind that the earlier work does not supply, and the criterion matters for the systems above, where a wrong sequence delays the most costly step rather than the average one.

In this paper we study the problem $F2 \parallel WC_{\max}$. Four features distinguish it from the settings reviewed above. (i) Two machines stand in series, so that each completion time depends on the idle time inherited from its predecessors. (ii) The objective is a maximum over weighted completion times rather than their sum. (iii) The processing times are general, with no proportionate assumption. (iv) No job may be rejected, so that every job must be

scheduled. Replacing the weighted sum by a weighted maximum, and dropping the proportionality assumption, would already put the problem beyond the single-machine reduction. We show that it is in fact strongly NP-hard (Theorem 3.2). The difficulty is structural rather than a matter of the instance size. Because the objective is determined by a single job, an exchange that shortens many completion times while lengthening that one does not improve the objective value. Because the jobs carry unequal weights, moreover, Johnson's rule has no direct extension to the weighted case.

Our results follow from the structure of an optimal schedule. Since the objective is a maximum, jobs of equal weight are interchangeable with respect to it. An optimal schedule can therefore be assumed to have a block structure, with one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. On this structure we build three methods, an exact dynamic program, a 2-approximation and an approximation scheme. The dynamic program runs in $O(nKV^{3K-3})$ time, where K is the number of distinct weights and V is the total processing time. It is practical when the weights fall into few classes, as in the plants described above. A 2-approximation runs in $O(n \log n)$ time, and rounding the input of the dynamic program gives a fully polynomial-time approximation scheme. The scheme attains a $(1+\varepsilon)$-approximation for constant K. For large instances the standard tools are constructive heuristics, of which NEH (Nawaz et al., 1983) is the reference, and genetic algorithms, whose operators and survival rules are adapted to the permutation encoding (Puka et al., 2024; Qiao et al., 2022). Because the exact methods remain exponential in K, we also develop two heuristics for large instances and report a computational study that compares them with the 2-approximation.

The rest of the paper is organized as follows. Section 2 fixes the notation, states the completion-time recurrence and gives the mixed integer programming formulation. Section 3 establishes the complexity of the problem. Section 4 proves the structural properties, develops the exact dynamic program, and presents the 2-approximation and the approximation scheme. Section 5 describes the two heuristics. Section 6 reports the computational study, and Section 7 concludes.

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$, the two-machine flow shop whose criterion is the maximum weighted completion time. The name is the three-field notation of scheduling theory (Pinedo, 2022): $F2$ denotes a flow shop in which every job passes through two machines in series, the second field is left empty because no further restriction is placed on the jobs, and $WC_{\max}$ denotes the criterion. The problem is referred to by this notation alone.

Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be n jobs processed on two machines M_1 and M_2 in series.

Each job J_j is processed first on M_1 for a_j time units and then on M_2 for b_j time units, without preemption, the processing times a_j and b_j being positive integers. Each machine processes at most one job at a time, and each job J_j carries a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the sequence on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative processing times on M_1 and M_2 after the first j jobs. The completion time of the job in position j on machine M_2 is

$$C_{\pi(j)} = \max_{1 \le i \le j} \Big\{ \sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)} \Big\}. \tag{1}$$

The objective is to find a schedule π that minimizes the largest of the products $w_{\pi(j)} C_{\pi(j)}$ over the positions,

$$WC_{\max}(\pi) = \max_{1 \le j \le n} w_{\pi(j)}\, C_{\pi(j)}. \tag{2}$$

Here C_j is the completion time of job J_j on M_2, and $C_{\pi(j)}$ is the completion time of the job in position j, given by (1). Throughout, $WC^*_{\max}$ denotes the optimal value of $F2 \parallel WC_{\max}$.

The problem can be formulated as a mixed integer linear program (MILP). The positions of a permutation schedule are indexed by $k = 1, \ldots, n$. We introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is assigned to position k, and 0 otherwise;
- C^1_k completion time of the job in position k on M_1;
- C^2_k completion time of the job in position k on M_2;
- C_j completion time of job J_j on M_2;
- Z the maximum weighted completion time $WC_{\max}$;
- M a sufficiently large constant, taken as $2 \sum_{j=1}^{n} (a_j + b_j)$.

The MILP model is:

$$\min \quad Z \tag{3}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, \ldots, n \tag{4}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, \ldots, n \tag{5}$$

$$C^1_1 = \sum_{j=1}^{n} a_j x_{j1} \tag{6}$$

$$C^1_k = C^1_{k-1} + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, \ldots, n \tag{7}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^{n} b_j x_{j1} \tag{8}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{9}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{10}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad j = 1, \ldots, n,\ k = 1, \ldots, n \tag{11}$$

$$Z \geq w_j C_j, \quad j = 1, \ldots, n \tag{12}$$

$$x_{jk} \in \{0, 1\}, \tag{13}$$

$$C_k^1, C_k^2, C_j \geq 0 \tag{14}$$

Constraints (4) and (5) assign every job to exactly one position and every position to exactly one job. Constraints (6) and (7) compute the completion times on M_1, and (8) to (10) those on M_2: for $k \geq 2$ the job in position k starts on M_2 only after it has completed M_1 and the preceding job has released M_2. Constraint (11) ties the position-based completion times to the job-based ones through a big-M formulation, so that $C_j \geq C_k^2$ when $x_{jk} = 1$, while the constraint is inactive when $x_{jk} = 0$, its right-hand side becoming $C_k^2 - M \leq 0$. Constraint (12) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$ of (2), and (13) and (14) require the assignment variables to be binary and the completion times nonnegative.

3 Computational complexity

Theorem 3.1. *$F2 \parallel WC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given r positive integers $x_1, \ldots, x_r$ with $\sum_{i=1}^{r} x_i = 2T$, do there exist disjoint subsets $Z_1, Z_2 \subseteq \{1, \ldots, r\}$ with $Z_1 \cup Z_2 = \{1, \ldots, r\}$ and

$$\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T?$$

We construct an instance with $n = r + 1$ jobs: for $j = 1, \ldots, n-1$ set $a_j = 1$, $b_j = r x_j$ and $w_j = T + 1$, and for the remaining job J_n set $a_n = rT$, $b_n = 1$ and $w_n = 2T + 1$. The question is whether some schedule satisfies

$$WC_{\max} \leq Y, \qquad \text{where} \qquad Y = r(T+1)(2T+1).$$

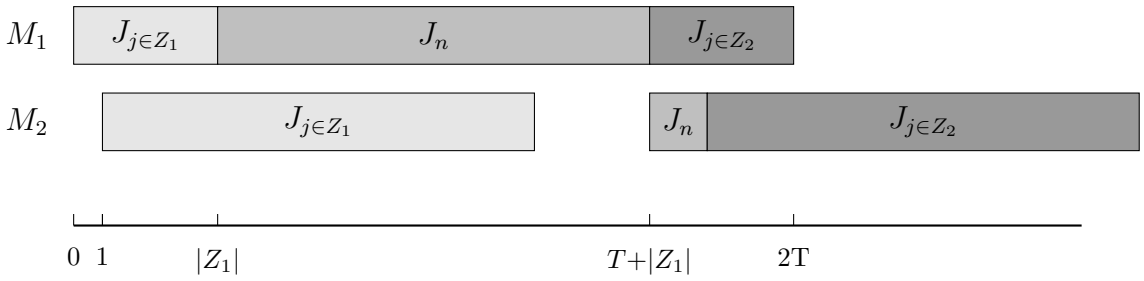


Figure 1: The reduction from Partition

Job J_n splits the remaining jobs into two subsets, and Figure 1 shows the layout the split leaves on the two machines. The M_2 workloads of the subsets are unknown, but they sum to $2rT$, so they can be written as $r(T+\xi)$ and $r(T-\xi)$ for some $-T \le \xi \le T$. The completion times of J_n are then

$$C_n^1 = |Z_1| + rT + 1, \tag{15}$$

$$C_n^2 = 1 + r(T+\xi) + 1, \tag{16}$$

and $C_n = \max\{C_n^1, C_n^2\}$.

If $\xi = 0$, the two subsets carry the same M_2 load, and the Partition instance has a solution. In this case C_n is given by (15), and

$$\begin{aligned} C_n &= |Z_1| + rT + 1 \le rT + r = r(T+1), \\ w_n C_n &= (2T+1)\, r(T+1) = Y, \\ WC_{\max} &= r(T+1)(C_n + rT) = Y, \end{aligned}$$

so $w_j C_j \le WC_{\max} = Y$.

Conversely, suppose that some schedule satisfies $WC_{\max} \le Y$. When $\xi > 0$, C_n is given by (16), and

$$\begin{aligned} C_n &= 1 + r(T+\xi) + 1 \ge 1 + r(T+1) + 1, \\ w_n C_n &\ge (2T+1)\big(1 + r(T+1) + 1\big) = Y. \end{aligned}$$

When $\xi < 0$, C_n is given by (15), and

$$\begin{aligned} C_{\max} &= C_n + \sum_{j\in Z_2} b_j \\ &= 1 + rT + |Z_1| + r(T-\xi) \\ &\ge rT + r + rT \\ &= r(2T+1) = Y, \\ WC_{\max} &\ge r(T+1)\big(r(2T+1)\big) = Y, \end{aligned}$$

so $WC_{\max} > Y$, a contradiction.

The Partition instance therefore admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \le Y$. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the strongly NP-complete 3-Partition problem: given $3m$ positive integers $x_1, \dots, x_{3m}$ and an integer B satisfying $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples, each of sum B?

Let $R = 3m + 5$, $D = RB + 4$, and $T = mD$. We construct an instance with $3m$ element jobs $J_1, \dots, J_{3m}$ and m separator jobs $S_1, \dots, S_m$:

- for every element job J_j, set $a_j = 1$, $b_j = Rx_j + 1$;
- for every separator job S_k, set $a_{S_k} = RB + 1$, $b_{S_k} = 1$.

All processing times are positive integers. For $k = 1, \dots, m$, define $d_k = kD + 1$ and let $Q = (T + 2)^3$. Set

$$w_{S_k} = \left\lfloor \frac{Q}{d_k} \right\rfloor, \qquad w_j = \left\lfloor \frac{Q}{T} \right\rfloor \quad (j = 1, \dots, 3m),$$

and use the decision threshold $Y = Q$. Since $d_k \le T + 1$ and $Q \ge d_k(d_k + 1)$, we have $\lfloor Q/d_k \rfloor > d_k$. Thus, for every $d \in \{d_1, \dots, d_m, T\}$ and every integer completion time C,

$$\left\lfloor \frac{Q}{d} \right\rfloor C \le Q \quad \Longrightarrow \quad C \le d. \tag{17}$$

Indeed, if $C \ge d + 1$, write $Q = qd + r$ with $q = \lfloor Q/d \rfloor$ and $0 \le r < d$. Then $q(d + 1) = Q - r + q > Q$ because $q > d > r$. Suppose first that the 3-Partition instance is a yes-instance. Let $G_1, \dots, G_m$ be triples with $\sum_{J_j \in G_k} x_j = B$ for every k; each triple contains exactly three elements, since two of its members sum to less than B and four to more. Schedule

$$G_1, S_1, G_2, S_2, \dots, G_m, S_m.$$

The three element jobs in G_k use 3 units on M_1, and S_k uses $RB + 1$ units, so S_k completes on M_1 at time kD. On M_2 the group G_k occupies the machine from time $(k - 1)D + 1$, which is the completion of S_{k-1} for $k \ge 2$ and that of the first element of the group on M_1 for $k = 1$, and it uses $RB + 3$ units there; the last element of G_k therefore completes at time kD. Thus

$$C_{S_k} = kD + 1 = d_k.$$

Every element job completes on M_2 no later than $T = mD$. By the definition of the weights, all weighted completion times are at most Q, so $WC_{\max} \le Y$.

Conversely, suppose that a schedule π satisfies $WC_{\max} \le Y$. Since the objective is regular, the standard permutation-schedule property for two-machine flow shops (Baker,

1974) lets us restrict attention to permutation schedules. The separator jobs have identical processing times, so if their positions are fixed, exchanging the labels of two separators that occur in the wrong order cannot increase the maximum weighted completion time: d_k rises with k, so $w_{S_1} \geq w_{S_2} \geq \cdots \geq w_{S_m}$, and the exchange only moves a larger weight to an earlier completion. We may consequently assume that the separators occur in the order $S_1, S_2, \ldots, S_m$. By (17),

$$C_{S_k} \leq kD + 1 \quad (k = 1, \ldots, m), \qquad C_j \leq T \quad (j = 1, \ldots, 3m).$$

No element job can occur after S_m. Indeed, if at least one element job did so, consider the last such element job. By the permutation-schedule property invoked above, the processing sequences on M_1 and M_2 coincide, so this job is also the last job on M_1. All m separators and all $3m$ element jobs therefore complete on M_1 no later than it, so its M_1 completion time is at least the total M_1 processing time

$$m(RB + 1) + 3m = m(RB + 4) = T.$$

Its M_2 completion time would then be strictly greater than T, contradicting $C_j \leq T$. The element jobs are therefore partitioned into the m groups G_k lying between S_{k-1} and S_k, where S_0 denotes the start of the schedule. Put

$$B_k = \sum_{J_j \in G_k} x_j, \qquad E_k = \sum_{h=1}^{k} |G_h|, \qquad E_0 = 0.$$

For $k = 2, \ldots, m$, the M_2 path gives

$$C_{S_k} \geq C_{S_{k-1}} + RB_k + |G_k| + 1, \tag{18}$$

while the M_1 path to S_{k-1} gives

$$C_{S_{k-1}} \geq (k - 1)(RB + 1) + E_{k-1} + 1. \tag{19}$$

Combining (18) and (19) with $C_{S_k} \leq kD + 1$ yields

$$R(B_k - B) + E_k \leq 3k, \qquad k = 2, \ldots, m. \tag{20}$$

For $k = 1$, the M_2 path and $C_{S_1} \leq D + 1$ similarly give

$$R(B_1 - B) + E_1 \leq 4. \tag{21}$$

Because $R = 3m + 5$, (20) and (21) imply $B_k \leq B$ for every k. Otherwise $B_k \geq B + 1$, and the respective left-hand side exceeds $3k$, or exceeds 4 when $k = 1$. Since every element job belongs to one of the groups, $\sum_{k=1}^{m} B_k = mB$, and hence $B_k = B$ for every k. Finally, $B/4 < x_j < B/2$ implies that a group with sum B contains exactly three elements: at most two elements have sum strictly less than B, while at least four elements have sum strictly greater than B. Thus $G_1, \ldots, G_m$ is a valid 3-partition.

The construction uses $4m$ jobs, and all processing times, weights and the threshold are bounded by a polynomial in m and B. Since 3-Partition remains NP-complete when B is polynomially bounded in m, this is a strong polynomial reduction, and $F2 \parallel WC_{\max}$ is strongly NP-hard. □

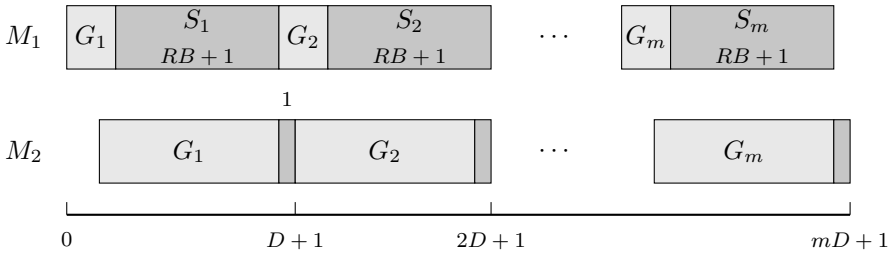


Figure 2: The reduction from 3-Partition

4 Exact and approximation algorithms

This section develops the exact dynamic program and the two approximation algorithms that rest on it. Throughout, K denotes the number of distinct weights and $V = \sum_{j=1}^{n}(a_j+b_j)$ the total processing time.

Since the objective is a maximum over weighted completion times, jobs of equal weight are interchangeable. An optimal schedule can therefore be assumed to have a block structure: one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. A schedule is fixed once each weight class has been assigned to a block, and the running time of every algorithm below is governed by the number K of those classes.

Three algorithms are built on this structure. The first is exact and prunes dominated states: a state records each block's processing time on both machines, the completion time of its last job on M_2, and that job's weight, and the dynamic program runs in $O(n \cdot K \cdot |\mathscr{H}|_{\max})$ time with $|\mathscr{H}|_{\max} \leq 2^K V^{3K-3}$. The second sorts the jobs by non-increasing weight in $O(n \log n)$ time and returns a schedule within a factor 2 of the optimum. The third rounds the time coordinates and the weights to geometrically spaced grids, which turns the dynamic program into a $(1+\epsilon)$-approximation scheme for a fixed number of distinct weights.

4.1 Block structure of an optimal schedule

For $F2 \parallel WC_{\max}$, an optimal schedule exists in which both machines process the jobs in the same order (Baker, 1974); we therefore restrict attention to such permutation schedules. Within this class, Johnson's rule (Johnson, 1954) minimizes the makespan: the jobs with $a_j \leq b_j$ are scheduled first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

Fix a permutation schedule π. Let $W_1 > W_2 > \cdots > W_K$ be the K distinct weights occurring among the jobs, and let

$$\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}, \qquad k = 1, \ldots, K,$$

denote the corresponding weight classes. Set $\ell_0 = 0$. For $k = 1, \ldots, K$ in sequence, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, that is, among $\pi(\ell_{k-1}+1), \ldots, \pi(n)$; if no such job remains, set $\ell_k = \ell_{k-1}$. The blocks are the consecutive segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1}+1), \ldots, \pi(\ell_k)\}, \qquad k = 1, \ldots, K,$$

so that in particular $\mathcal{B}_1 = \{\pi(1), \ldots, \pi(\ell_1)\}$. Equivalently, once the blocks $\mathcal{B}_1, \ldots, \mathcal{B}_k$ have been removed, the remaining jobs are

$$\mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}_i = \{\pi(\ell_k+1), \ldots, \pi(n)\}, \qquad k = 1, \ldots, K.$$

Hence $\ell_1 \le \ell_2 \le \cdots \le \ell_K$, and no block extends beyond its own last job. For each block $\mathcal{B}_i$, let A_i and B_i denote the total processing time of its jobs on M_1 and M_2, respectively, C_i the completion time of its last job on M_2, and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job; such a block is called empty and satisfies $A_k = B_k = C_k = L_k = 0$, and it may host any job of weight at most W_k. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight W_k, whereas an empty block has $L_k = 0$. Since $W_1 > W_2 > \cdots > W_K$, the block-end weights are non-increasing on the non-empty blocks.

We illustrate the block construction with a small example.

Example 4.1. *Consider a six-job instance with sequence $\pi = (J_1, \ldots, J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The weight classes are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, and $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, and the resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, and $\mathcal{B}_3 = \{J_6\}$.

Lemma 4.1. *For any schedule π, scheduling the jobs of each block by Johnson's rule (Johnson, 1954) and concatenating the blocks in their original order gives a schedule π' with $WC_{\max}(\pi') \le WC_{\max}(\pi)$.*

Proof. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight L_k, and every job in $\mathcal{B}_k$ has weight at most L_k: a weight occurring in $\mathcal{B}_k$ either equals W_k, or it has already

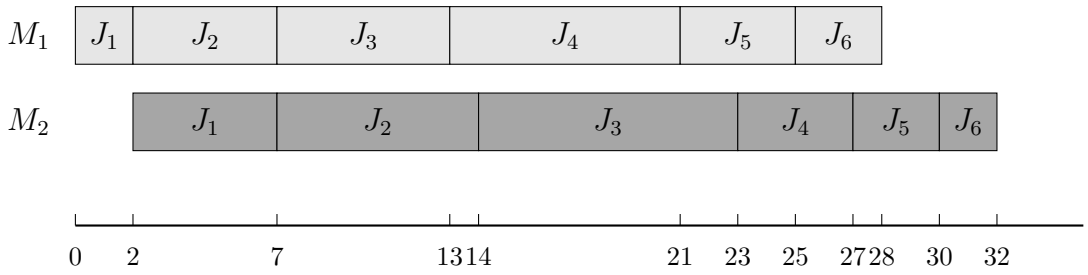


Figure 3

appeared in an earlier block, whose weights are larger. Let $C_{\pi(\ell_k)}$ denote the completion time of the last job of $\mathcal{B}_k$ in π, the job that sits at position ℓ_k. An empty block satisfies $L_k C_{\pi(\ell_k)} = 0$ and therefore contributes nothing to the objective, so the argument below applies to the non-empty blocks.

We first show that the maximum weighted completion time within a block is attained by the last job of that block. For any $J_j \in \mathcal{B}_k$, the last job of the block completes no earlier than J_j in π, so $C_j \le C_{\pi(\ell_k)}$, while $w_j \le L_k$ by the block construction. Hence $w_j C_j \le L_k C_{\pi(\ell_k)}$, and taking the maximum over $j \in \mathcal{B}_k$ gives

$$\max_{J_j \in \mathcal{B}_k} w_j C_j = L_k C_{\pi(\ell_k)}.$$

The objective of (2) therefore equals

$$WC_{\max}(\pi) = \max_{k=1,\dots,K} L_k C_{\pi(\ell_k)},$$

so it depends only on the last job of each block.

Let π' be the schedule obtained by rescheduling the jobs within each block $\mathcal{B}_k$ according to Johnson's rule, keeping the blocks in their original order. We prove $C_{\pi'(\ell_k)} \le C_{\pi(\ell_k)}$ for every k by induction on k.

Base case $k = 1$. Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal order, so Johnson's rule gives $C_{\pi'(\ell_1)} \le C_{\pi(\ell_1)}$.

Inductive step. Assume $C_{\pi'(\ell_{k-1})} \le C_{\pi(\ell_{k-1})}$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \dots, \mathcal{B}_{k-1}$, which is invariant under internal rescheduling, so $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2, block $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\, C_{\pi(\ell_{k-1})}\}$; the first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π. Together with the same M_1 start and the internally optimal Johnson scheduling, this gives $C_{\pi'(\ell_k)} \le C_{\pi(\ell_k)}$, which completes the induction.

Finally,

$$WC_{\max}(\pi') = \max_k L_k C_{\pi'(\ell_k)} \le \max_k L_k C_{\pi(\ell_k)} = WC_{\max}(\pi),$$

which establishes the lemma. □

4.2 An exact dynamic program

The exact dynamic program processes the jobs in the Johnson sequence and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program. Given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in the order of non-increasing block weights yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each $j = 1, \ldots, n$, let $\mathscr{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records the following quantities for every block $\mathcal{B}_i$:

- A_i: the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i: the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i: the completion time of the last job of block $\mathcal{B}_i$ on machine M_2;
- L_i: the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix. Writing

$$i^* = \max\{\, i : w_j \le W_i \,\},$$

the eligible set is $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. Each state is therefore the aggregate K-block tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

which is described by $4K$ parameters.

The initial state set contains a single state whose components are all zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. In Algorithm 1, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2.

Lemma 4.2. *Let $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$ and $\mathscr{H}' = (A'_1, \ldots, A'_K; B'_1, \ldots, B'_K; C'_1, \ldots, C'_K; L'_1, \ldots, L'_K)$ be two states in $\mathscr{H}_j$. If $A_i = A'_i$, $B_i = B'_i$ and $L_i = L'_i$ for every i, and $C_i \le C'_i$ for every i, then $\mathscr{H}'$ may be discarded without changing the optimal value.*

Proof. Consider any continuation that extends $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that assigns the remaining jobs identically. For every block $\mathcal{B}_k$ that follows the updated one, the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, and C_{k-1} is no larger under $\mathscr{H}$ than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$. Since A_i, B_i and L_i are identical in the two states, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ by placing J_j at the end of each eligible block. That is, for every state in $\mathscr{H}_{j-1}$ and every $i \in \mathcal{E}(J_j)$, only the i-th slice of the state is updated while all other blocks remain unchanged:

$$A_i' = A_i + a_j, \quad B_i' = B_i + b_j, \quad C_i' = \max\{C_i + b_j,\ A_i + a_j + b_j\}. \tag{22}$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

Algorithm 1: Block concatenation and evaluation

Input: State $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$.

Step 1. [Initialization] Set $T_{A_0} \leftarrow 0$, $T_{B_0} \leftarrow 0$, $WC_{\max} \leftarrow 0$.
Step 2. [Evaluation] **for** $i = 1$ **to** K **do**
  $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
  $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
  if $L_i > 0$ **then**
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
  end
end

Output: $WC_{\max}$.

To support Step 4, each state in $\mathscr{H}_j$ stores a pointer to the state in $\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from a state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers the block $\mathcal{B}_i$ containing each job J_j. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves $F2 \parallel WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \le 2^K V^{3K-3}$ and $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. For constant K, this is $O(nV^{3K-3})$.*

Proof. Each state in $\mathscr{H}_j$ is the $4K$-tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

so it suffices to bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of the jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0,\ V]$. Since the j jobs considered so far form a prefix of the fixed Johnson sequence, $\sum_{i=1}^{K} A_i$ equals the sum of the a_h over that prefix. At most $K - 1$ of the A_i are therefore independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^{K} B_i$ is the prefix sum of the b_j, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when that block is scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le V$, the C-vector contributes at most V^{K-1} distinct

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; weights $W_1 > \cdots > W_K$.

Step 1. [Initialization] Set $L_i \leftarrow 0$ for $i = 1, \ldots, K$, set $\mathscr{H}_0 \leftarrow \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$ and $Z^* \leftarrow +\infty$.

Step 2. [Generation] **for** $j = 1$ **to** n **do**

  for *each* $(A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_{j-1}$ **do**

    for *each block* $i \in \mathcal{E}(J_j) = \{1, \ldots, i^*\}$ **do**

      $A'_i \leftarrow A_i + a_j$, $B'_i \leftarrow B_i + b_j$, $L'_i \leftarrow w_j$

      $C'_i \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$

      Add the state $(A_1, \ldots, A'_i, \ldots, A_K; B_1, \ldots, B'_i, \ldots, B_K; C_1, \ldots, C'_i, \ldots, C_K; L_1, \ldots, L'_i, \ldots, L_K)$ to $\mathscr{H}_j$

    end

  end

end

[Elimination] Elimination $\mathscr{H}_j$ for $j \in \{1, \ldots, n\}$ by Lemma 4.2

Step 3. [Calculate] **for** *each state* $S = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_n$ **do**

  if $L_i \in \{0, W_i\}$ *for every* $i = 1, \ldots, K$ **then**

    $WC_{\max} \leftarrow$ Algorithm 1 applied to S

    $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$

  else

    Discard S.

  end

end

Step 4. [Result] Recover a schedule attaining Z^* by standard backtracking.

Output: An optimal schedule π^*.

values. The weight L_i equals W_i when block $\mathcal{B}_i$ is non-empty and 0 when it is empty, so the L-vector takes at most 2^K distinct values. Multiplying these bounds gives

$$|\mathscr{H}_j| \le 2^K \cdot V^{K-1} \cdot V^{2K-2} = 2^K \cdot V^{3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \ldots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Iteration j therefore costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over the n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation by Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. For fixed K, 2^K is constant and the bound becomes $O(n \cdot V^{3K-3})$. □

4.3 A 2-approximation

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

1 **Step 1.** Sort the jobs by non-increasing weight w_j, breaking ties by non-increasing $a_j + b_j$.

2 **Step 2.** Schedule the jobs in this sequence on both machines.

Output: The resulting permutation schedule π.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \le 2\, WC^*_{\max}$.*

Proof. Let π be the schedule returned by Algorithm 3. Its jobs appear in non-increasing order of weight, so we name them $J_1, J_2, \dots, J_n$, with $w_1 \ge w_2 \ge \cdots \ge w_n$ and $\pi = (J_1, J_2, \dots, J_n)$. Every job of the prefix $S = \{J_1, J_2, \dots, J_j\}$ then carries a weight of at least w_j.

We first bound the completion times in π. By (1), the job J_j, which sits at position j of π, completes on M_2 at time $C_{\pi(j)}$, and every term of that maximum is bounded by the full sum:

$$\sum_{h=1}^{i} a_h + \sum_{h=i}^{j} b_h \le \sum_{h=1}^{j} a_h + \sum_{h=1}^{j} b_h = \sum_{h \in S} \big(a_h + b_h\big), \qquad 1 \le i \le j.$$

Hence

$$C_{\pi(j)} \;\le\; \sum_{h \in S} \big(a_h + b_h\big). \tag{23}$$

Since $WC_{\max}(\pi) = \max_{1 \le j \le n} w_{\pi(j)}\, C_{\pi(j)}$ ((2)), and $w_{\pi(j)} = w_j$ for the job at position j, multiplying (23) by $w_j > 0$ and taking the maximum over j gives

$$WC_{\max}(\pi) = \max_{1 \le j \le n} w_{\pi(j)}\, C_{\pi(j)} \le \max_{1 \le j \le n} w_j \sum_{h \in S} \big(a_h + b_h\big). \tag{24}$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix $S = \{J_1, J_2, \dots, J_j\}$ and let π^* be an optimal schedule. Let J_u be the job of S that finishes last on M_1 in π^*, and J_v the job of S that finishes last on M_2. All jobs of S are processed on M_1 before J_u completes M_1, so $C^*_u \ge \sum_{h \in S} a_h$. In the same way, all M_2 processing of the jobs of S precedes the completion of J_v on M_2, so $C^*_v \ge \sum_{h \in S} b_h$. Since both J_u and J_v lie in S, their weights satisfy $w_u, w_v \ge w_j$. The value $WC^*_{\max}$ bounds the weighted completion time of every job of π^*, so $WC^*_{\max} \ge w_u C^*_u \ge w_j \sum_{h \in S} a_h$ and $WC^*_{\max} \ge w_v C^*_v \ge w_j \sum_{h \in S} b_h$.

Since the prefix S was arbitrary, these bounds hold for every prefix, and a single job bounds the optimum on its own as well. Three lower bounds therefore follow,

$$LB_1 = \max_{1 \le j \le n} w_j \sum_{h \in S} a_h, \tag{25}$$

$$LB_2 = \max_{1 \le j \le n} w_j \sum_{h \in S} b_h, \tag{26}$$

$$LB_3 = \max_{1 \le j \le n} w_j \, (a_j + b_j), \tag{27}$$

We call them the M_1 threshold LB_1, the M_2 threshold LB_2 and the job threshold LB_3. The first two charge the workload of the prefix to the job of the prefix that leaves its machine last. That job carries a weight of at least w_j, and it completes its machine only after the whole workload of S has been processed there. Hence $WC^*_{\max} \ge w_j \sum_{h \in S} a_h$ and $WC^*_{\max} \ge w_j \sum_{h \in S} b_h$. The job threshold charges one job its own two operations, which run one after the other and neither of which can be left out, so $C^*_j \ge a_j + b_j$. As J_j carries the weight w_j itself, $w_j(a_j + b_j)$ is a lower bound on the optimum as well. Each bound is determined by the instance alone, and their largest is the reference value of the instance,

$$LB = \max\{LB_1, LB_2, LB_3\}, \tag{28}$$

so that $LB \le WC^*_{\max}$.

For the prefix S, the two bounds give $w_j \sum_{h \in S}(a_h + b_h) \le LB_1 + LB_2 \le 2\,LB \le 2\,WC^*_{\max}$, and the claim follows from (24). □

4.4 A fully polynomial-time approximation scheme

4.4.1 Rounding the time coordinates

The exact dynamic program of Algorithm 2 becomes a fully polynomial-time approximation scheme once the input data are rounded (Hall, 1994). The technique rests on bounds for the coordinates that are rounded: each time coordinate ranges over a bounded interval, and rounding it to one of a fixed number of geometrically spaced values keeps the relative error within a factor θ. The total processing times A_i, B_i of block $\mathcal{B}_i$ on M_1 and M_2, and its completion time C_i, are rounded. The weight L_i of the last job of block $\mathcal{B}_i$ is not rounded; following Algorithm 2, the positional constraint $L_i \in \{0, W_i\}$ is required for every block, and a state violating it is discarded.

Fix $0 < \epsilon < 1$ and let

$$\theta = 1 + \frac{\epsilon}{2n}, \qquad V_A = \sum_{j=1}^{n} a_j, \qquad V_B = \sum_{j=1}^{n} b_j, \qquad V = V_A + V_B.$$

For every block $\mathcal{B}_i$, the recurrence (22) preserves $A_i \le V_A$ and $B_i \le V_B$, since block $\mathcal{B}_i$ contains a subset of the jobs; moreover $C_i \le A_i + B_i \le V$. We therefore round the A

dimension over $[0, V_A]$, the B dimension over $[0, V_B]$, and the C dimension over $[0, V]$. To this end, let

$$\Delta(x) = k = \lfloor \log_\theta x \rfloor \quad \text{for every } x \geq 1 \text{ with } \theta^k \leq x < \theta^{k+1}, \qquad \text{and} \qquad \Delta(0) = -1.$$

All processing times are integral, so every value taken by a coordinate A_i, B_i, C_i is a non-negative integer. Hence any two values $y, y' \geq 1$ with $\Delta(y) = \Delta(y')$ satisfy $y \leq \theta\, y'$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

The *label* of a state $\mathscr{H}^\epsilon = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$ is the tuple formed by applying Δ to its time coordinates, together with its block-end weights,

$$\ell(\mathscr{H}^\epsilon) = \big(\Delta(A_1), \ldots, \Delta(A_K);\ \Delta(B_1), \ldots, \Delta(B_K);\ \Delta(C_1), \ldots, \Delta(C_K);\ L_1, \ldots, L_K\big).$$

Thus two states share a label exactly when every coordinate A_i, B_i and C_i falls into the same box and their block-end weights coincide. Each dimension $x \in \{A, B, C\}$ contributes at most $v^x + 2$ distinct values of Δ, where $v^x = \lceil \log_\theta V_x \rceil$ and $V_C = V$, while the weight is not rounded and each L_i takes at most $n + 1$ values.

The Elimination step keeps one representative per label. At iteration j, every state of $\mathscr{H}^\epsilon_{j-1}$ spawns one candidate for each eligible block $i \in \mathcal{E}(J_j) = \{1, \ldots, i^*\}$, and each candidate is assigned its label. Two candidates with the same label are indistinguishable after rounding and carry the same block-end weights $L_1, \ldots, L_K$, so the step keeps any one of them and discards the rest: the kept state satisfies $x \leq \theta\, \tilde{x}$ in every time coordinate $x \in \{A, B, C\}$ of every block, relative to every candidate $\tilde{\mathscr{H}}^\epsilon$ it replaces, while its block-end weights are unchanged. Every label therefore carries exactly one state into $\mathscr{H}^\epsilon_j$, and the number of states never exceeds the number of labels.

Lemma 4.3. *Let $\mathscr{H}_j = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$, $j = 1, \ldots, n$, be the states along an optimal trajectory of Algorithm 2, the states that backtracking from an optimal final state recovers. For every j, Algorithm 4 keeps a state $\hat{\mathscr{H}}^\epsilon_j = (\hat{A}_1, \ldots, \hat{A}_K; \hat{B}_1, \ldots, \hat{B}_K; \hat{C}_1, \ldots, \hat{C}_K; L_1, \ldots, L_K)$ that carries the same block-end weights $L_1, \ldots, L_K$ and satisfies*

$$\hat{A}_i \leq A_i\, \theta^{j-1}, \quad \hat{B}_i \leq B_i\, \theta^{j-1}, \quad \hat{C}_i \leq C_i\, \theta^{j-1}, \quad (1 \leq i \leq K). \tag{29}$$

Proof. We argue by induction on j, from 1 to n.

Base case $j = 1$. Both algorithms generate their candidates for J_1 from the same zero state. Two candidates that place J_1 in different blocks differ in at least one time coordinate, so they carry different labels and no state is deleted. The sets $\mathscr{H}^\epsilon_1$ and $\mathscr{H}_1$ therefore coincide, and (29) holds with equality at $j = 1$.

Inductive step $j \geq 2$. Assume that (29) holds at $j - 1$, and let i be the block to which $\mathscr{H}_{j-1}$ appends J_j in Algorithm 2. Algorithm 2 assigns a job only to a block i with $w \leq W_i$,

Algorithm 4: State-class FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; accuracy ϵ.

Step 1. [Initialization] Set $\theta = 1 + \epsilon/(2n)$; set $Z^* \leftarrow +\infty$;
$\mathscr{H}_0^\epsilon \leftarrow \{(0,\dots,0;\ 0,\dots,0;\ 0,\dots,0;\ 0,\dots,0)\}$.
Step 2. [Generation] **for** $j = 1$ **to** n **do**
  for *each state* $(A_1,\dots,A_K; B_1,\dots,B_K; C_1,\dots,C_K; L_1,\dots,L_K) \in \mathscr{H}_{j-1}^\epsilon$ **do**
    for *each block* $i \in \mathcal{E}(J_j) = \{1,\dots,i^*\}$ **do**
      $\tilde{A}_i = A_i + a_j$, $\tilde{B}_i = B_i + b_j$, $\tilde{L}_i = w_j$
      $\tilde{C}_i = \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
      [Label] Record the label $\ell(\tilde{\mathscr{H}}_j^\epsilon) =$
        $\big(\Delta(\tilde{A}_1),\dots,\Delta(\tilde{A}_K);\ \Delta(\tilde{B}_1),\dots,\Delta(\tilde{B}_K);\ \Delta(\tilde{C}_1),\dots,\Delta(\tilde{C}_K);\ \tilde{L}_1,\dots,\tilde{L}_K\big)$.
    end
    [Elimination] Among the candidate states sharing the same label, keep one of them and delete the others; set $\mathscr{H}_j^\epsilon$ to the kept states.
  end
end
Step 3. [Calculate] Same as Step 3 of Algorithm 2, applied to $\mathscr{H}_n^\epsilon$.
Step 4. [Result] Same as Step 4 of Algorithm 2.

Output: An approximate schedule π.

so $w_j \le W_i$. This constraint depends only on J_j and the block index, hence appending J_j to block i is available from $\hat{\mathscr{H}}_{j-1}^\epsilon$ as well. Write A'_i, B'_i and C'_i for the coordinates of $\mathscr{H}_{j-1}$. Applying the exact recurrences to $\hat{\mathscr{H}}_{j-1}^\epsilon$ and using $\theta^{j-2} \ge 1$ gives a candidate $\tilde{\mathscr{H}}_j^\epsilon$ with

$$\begin{aligned}
\tilde{A}_i &= \hat{A}_i + a_j \le \big(A'_i + a_j\big)\theta^{j-2} = A_i\theta^{j-2},\\
\tilde{B}_i &= \hat{B}_i + b_j \le \big(B'_i + b_j\big)\theta^{j-2} = B_i\theta^{j-2},\\
\tilde{C}_i &= \max\big\{\hat{C}_i + b_j,\ \hat{A}_i + a_j + b_j\big\}\\
&\le \max\big\{C'_i + b_j,\ A'_i + a_j + b_j\big\}\theta^{j-2} = C_i\theta^{j-2},
\end{aligned}$$

where A_i, B_i and C_i are the coordinates of $\mathscr{H}_j$. For the remaining blocks $m \ne i$ the same transition leaves the coordinates unchanged, so that $\tilde{x}_m = \hat{x}_m \le x'_m\theta^{j-2} = x_m\theta^{j-2}$ for $x \in \{A, B, C\}$. Hence $\tilde{x}_m \le x_m\theta^{j-2}$ holds for every block $m = 1,\dots,K$. The block-end weights $L_1,\dots,L_K$ of the candidate are those of $\mathscr{H}_j$, because $L_i = w_j$ and the remaining block-end weights are unchanged.

The candidate $\tilde{\mathscr{H}}_j^\epsilon$ carries the label $\ell(\tilde{\mathscr{H}}_j^\epsilon)$ assigned in Algorithm 4, and the algorithm keeps a state $\hat{\mathscr{H}}_j^\epsilon \in \mathscr{H}_j^\epsilon$ with that same label. Since the label records the block-end weights, $\hat{\mathscr{H}}_j^\epsilon$ carries $L_1,\dots,L_K$ as well. Interval dominance states that two values with the same index Δ differ by at most a factor θ, so $y_m^x \le \theta\,\tilde{x}_m$ for every $x \in \{A, B, C\}$ and every block $m = 1,\dots,K$, where y_m^x denotes the coordinates of $\hat{\mathscr{H}}_j^\epsilon$. Together with $\tilde{x}_m \le x_m\theta^{j-2}$, this yields (29) at j and completes the induction.

Every state of $\mathscr{H}_n^\epsilon$ stores all $3K$ time coordinates, so the induction covers the coordinates of the last block $\mathcal{B}_K$ as well. Storing these coordinates for every block, exactly as in Algorithm 2, rather than deriving them from the others, preserves the factor θ^{j-1} of (29): a derived value would accumulate the rounding errors of the remaining blocks. □

Theorem 4.3. *For every $0 < \epsilon < 1$, Algorithm 4 returns a schedule with*

$$WC_{\max} \;\le\; (1+\epsilon)\, WC^*_{\max}.$$

For fixed K, it runs in $O\Big(n^{4K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\epsilon^{3K}}\Big)$ time, which is polynomial in n, $\log V$ and $1/\epsilon$.

Proof. We first bound the value of the returned schedule, and then its running time.

Let $\mathscr{H}_n$ be the final state of the optimal trajectory of Algorithm 2, and let $\hat{\mathscr{H}}_n^\epsilon \in \mathscr{H}_n^\epsilon$ be the state guaranteed by Lemma 4.3 at $j = n$. By the lemma, the time coordinates of $\hat{\mathscr{H}}_n^\epsilon$ are within a factor θ^{n-1} of those of $\mathscr{H}_n$, and its block-end weights are those of $\mathscr{H}_n$, because the weights are not rounded. The final state $\mathscr{H}_n$ satisfies the positional constraint $L_i \in \{0, W_i\}$ of Algorithm 2, so every block of $\hat{\mathscr{H}}_n^\epsilon$ has block-end weight at most W_i.

Let $\hat{\pi}$ and π^* be the schedules obtained by concatenating the blocks of $\hat{\mathscr{H}}_n^\epsilon$ and $\mathscr{H}_n$, and let $\hat{C}_i$ and C_i denote the completion times of the last job of block $\mathcal{B}_i$ in the two schedules, which are the corresponding coordinates of the two states. By the concatenation recurrence of Algorithm 1, C_i is nondecreasing in every coordinate of the state and positively homogeneous. Hence $\hat{C}_i \le \theta^{n-1} C_i$ for every i. Since every block of $\hat{\mathscr{H}}_n^\epsilon$ has block-end weight at most W_i, Lemma 4.1 yields

$$WC_{\max}(\hat{\pi}) = \max_{1\le i\le K} W_i\, \hat{C}_i \le \theta^{n-1} \max_{1\le i\le K} W_i\, C_i = \theta^{n-1}\, WC^*_{\max}.$$

The factor θ^{n-1} is at most $1+\epsilon$ for $0 < \epsilon < 1$. Because $1 + x \le e^x$ for every real x,

$$\theta^{n-1} = \Big(1 + \frac{\epsilon}{2n}\Big)^{n-1} \le e^{\frac{(n-1)\epsilon}{2n}} < e^{\epsilon/2},$$

where the strict inequality uses $(n-1)/(2n) < 1/2$. It is therefore enough to check that $e^{\epsilon/2} \le 1+\epsilon$ on $[0,1]$. The function $g(\epsilon) = 1 + \epsilon - e^{\epsilon/2}$ satisfies $g(0) = 0$ and

$$g'(\epsilon) = 1 - \tfrac{1}{2} e^{\epsilon/2} \ge 1 - \tfrac{1}{2} e^{1/2} > 0 \qquad (0 \le \epsilon \le 1),$$

because $e^{\epsilon/2} \le e^{1/2} < 2$ on that interval. Hence g is increasing, so $g(\epsilon) \ge 0$ and $e^{\epsilon/2} \le 1+\epsilon$. Consequently $\theta^{n-1} \le 1+\epsilon$, and the schedule returned by Algorithm 4, being no worse than $\hat{\pi}$, has a value of at most $(1+\epsilon)\, WC^*_{\max}$.

We now bound the running time. At each level j, Algorithm 4 keeps at most one state per label, and the number of labels is $(v^A+2)^K (v^B+2)^K (v^C+2)^K (n+1)^K$. The rounded time coordinates contribute $(v^A+2)^K (v^B+2)^K (v^C+2)^K$, which is at most $(v^C+2)^{3K}$ since $v^A, v^B \le v^C$, and each of the K block-end weights takes at most $n+1$ values. Each state generates

at most K candidates, one per eligible block, in $O(1)$ time each. The final evaluation of the states of $\mathscr{H}^{\epsilon,n}$ costs $O(K)$ per state. The running time is therefore $O\big(nK\,(v^C{+}2)^{3K}(n{+}1)^K\big)$. Since $\ln(1+\epsilon/(2n)) \geq \epsilon/(4n)$ for $0<\epsilon<1$, we have

$$v^C = \left\lceil \frac{\ln V}{\ln \theta} \right\rceil = O\!\left(\frac{n \log V}{\epsilon}\right).$$

For constant K, the bound reduces to $O(n^{4K+1}K(\log V)^{3K}/\epsilon^{3K})$, which is polynomial in n, $\log V$ and $1/\epsilon$. □

4.4.2 Rounding the weights

The FPTAS above rounds the time coordinates A_i, B_i, C_i and assumes that the number K of distinct weights is fixed. An analogous geometric rounding applies to the weights themselves when their ratio is bounded. Let $W_{\min} = \min_j w_j$, $W_{\max} = \max_j w_j$, and suppose $W_{\max}/W_{\min} \leq c$ for a constant c. Partition the interval $[W_{\min}, W_{\max}]$ into

$$J_0 = [W_{\min}, W_{\min}\theta), \qquad J_r = [W_{\min}\theta^r, W_{\min}\theta^{r+1}) \;\; (1 \leq r \leq q-1), \qquad J_q = [W_{\min}\theta^q, W_{\max}],$$

with $q = \big\lceil \log_\theta(W_{\max}/W_{\min}) \big\rceil$, and round each weight w_j upward to the right endpoint of its interval,

$$\hat{w}_j = W_{\min}\,\theta^{\,r_j+1}, \qquad r_j = \left\lfloor \log_\theta \frac{w_j}{W_{\min}} \right\rfloor,$$

so that $w_j \leq \hat{w}_j \leq \theta\, w_j$. The rounded weights take only $q+1 = O\big(\log_\theta(W_{\max}/W_{\min})\big)$ distinct values, so jobs whose weights differ by a factor at most θ merge into a single class.

Lemma 4.4. *Let $\hat{w}_j$ be the rounded weights defined above, and let $\hat{\pi}$ be an optimal schedule of the instance with weights $\hat{w}_j$, whose optimal value is $\hat{W}C^*_{\max}$. Then*

$$WC_{\max}(\hat{\pi}) \leq \theta\, WC^*_{\max}. \tag{30}$$

Proof. By construction $w_j \leq \hat{w}_j \leq \theta w_j$ for every job J_j. By (2) and the monotonicity of the maximum, every schedule π satisfies

$$WC_{\max}(\pi) = \max_{1\leq j\leq n} w_{\pi(j)}C_{\pi(j)} \leq \max_{1\leq j\leq n} \hat{w}_{\pi(j)}C_{\pi(j)} = \hat{W}C_{\max}(\pi) \leq \theta\, WC_{\max}(\pi).$$

Let π^* be an optimal schedule of the original instance. The rounded problem is solved optimally by $\hat{\pi}$, and π^* is feasible for it, so

$$WC_{\max}(\hat{\pi}) \leq \hat{W}C_{\max}(\hat{\pi}) = \hat{W}C^*_{\max} \leq \hat{W}C_{\max}(\pi^*) \leq \theta\, WC_{\max}(\pi^*) = \theta\, WC^*_{\max},$$

which is (30). □

Hence the weight rounding incurs a relative error of at most $\theta = 1+\epsilon/(2n) \leq 1+\epsilon$, exactly as in the completion-time rounding.

5 Heuristic algorithms

The exact dynamic program of Section 4 is pseudo-polynomial, and the running time of the FPTAS of Section 4.4 is exponential in the number K of distinct weights. Fast heuristics are therefore the practical route for large instances. This section presents two of them for $F2 \parallel WC_{\max}$: an adaptation of the NEH algorithm (Puka et al., 2024) and a genetic algorithm (Qiao et al., 2022).

Throughout this section, the objective of a permutation $\pi = (\pi(1), \ldots, \pi(k))$ of k jobs is evaluated by (1) and (2), so that

$$WC_{\max}(\pi) = \max_{1 \le t \le k} w_{\pi(t)} C_{\pi(t)},$$

which is computed in $O(k)$ time.

5.1 The NEH heuristic

Constructive heuristics are attractive for $F2 \parallel WC_{\max}$ because they produce a complete schedule in a single pass, without the tuning effort of population-based metaheuristics. The NEH heuristic (Puka et al., 2024) is among the most effective constructive rules for permutation flow shops, and its core idea is to build a schedule by incrementally inserting jobs into a growing partial sequence, each time choosing the position that best serves the objective.

In the maximum weighted completion-time objective, two properties are particularly important. First, jobs with large total processing time $a_j + b_j$ tend to push the whole makespan forward and therefore should appear early so as not to delay other jobs. Second, given a fixed makespan, jobs with larger weight w_j benefit more from occupying earlier positions, because their weighted completion time decreases. These two properties suggest a greedy priority in which large-$a_j + b_j$ jobs are placed first and large-w_j jobs are used as a tie-breaker.

We adapt the NEH framework as follows. The jobs are first sorted by non-increasing $a_j + b_j$, breaking ties by non-increasing w_j, to obtain an initial sequence $J_1, \ldots, J_n$. The schedule starts with the single job J_1. For each $k = 2, \ldots, n$, the next job J_k is inserted into every one of the k possible positions of the current partial schedule. Each candidate sequence is evaluated by (1), and the position that yields the smallest $WC_{\max}$ is retained. The pseudo-code is given in Algorithm 5.

Because the partial schedule already contains $k - 1$ jobs, the cost of evaluating all k insertions at step k is $O(k^2)$ using the recurrence (1); the total running time is therefore $O(n^3)$ (Theorem 5.1). The heuristic also produces a feasible permutation schedule by construction and requires no hyperparameter selection, so it serves as a fast standalone solver for large instances.

Theorem 5.1. *Algorithm 5 runs in $O(n^3)$ time.*

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. [Sequence] Sort the jobs by non-increasing $a_j + b_j$, breaking ties by w_j; let $J_1, \ldots, J_n$ be the resulting sequence.

Step 2. [Initialization] $\pi \leftarrow (J_1)$.

Step 3. [Insertion] **for** $k = 2$ **to** n **do**

  $\pi \leftarrow$ the sequence obtained by inserting J_k into π at a position minimizing $WC_{\max}$.

end

Output: Schedule π.

Proof. At step k, the job J_k is inserted into each of the k positions of a partial schedule of $k-1$ jobs, and each candidate sequence is evaluated in $O(k)$ time by the recurrence above. Hence step k costs $O(k^2)$, and the total running time is $O\big(\sum_{k=1}^{n} k^2\big) = O(n^3)$. □

5.2 The genetic algorithm

The objective $WC_{\max}$ is a bottleneck: a single *critical* job attains the maximum and fixes the value of a schedule. An ordering can therefore be improved in only two ways, by lowering the completion time of that job or by shortening completion times across the board. High-weight jobs should occupy early positions, so that their weighted completion stays small, while the makespan should stay low so that no job is delayed unnecessarily, and a genetic algorithm for $F2 \parallel WC_{\max}$ cannot treat the jobs symmetrically.

The genetic algorithm is a standard tool for flow-shop scheduling (Qiao et al., 2022), and it is used here within its classical framework, organised in four parts: the encoding and decoding of a schedule, the generation of the initial population, the iteration of the population, and the mutation of the offspring. Nothing in that framework is built around the weight classes. The population starts from the order that is shortest on the two machines and knows nothing of the weights, and it is the mutation that moves the blocks of one weight, one block at a time, until the schedule carries the structure of Lemma 4.1.

5.2.1 Encoding and decoding

A chromosome is a permutation $\pi = (\pi(1), \ldots, \pi(n))$, in which the gene at position t is the job scheduled t-th. Permutation encoding always represents a feasible permutation schedule, because every job occurs exactly once. No repair operator is therefore needed to restore feasibility after crossover or mutation. To decode a chromosome, both machines process the jobs in the order $\pi(1), \ldots, \pi(n)$. The completion time $C_{\pi(t)}$ of the t-th job follows from the closed form (1), which is evaluated in $O(n)$ time using the prefix sums

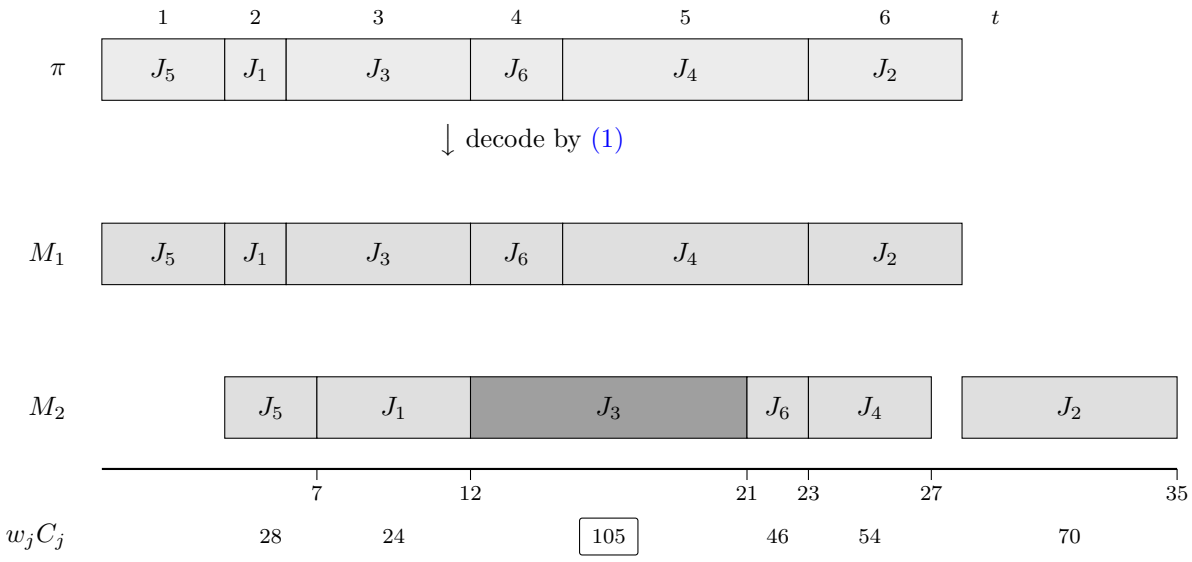


Figure 4: The chromosome $\pi = (J_5, J_1, J_3, J_6, J_4, J_2)$ and the schedule it decodes to, with w_jC_j below each job

$A_t = \sum_{h=1}^{t} a_{\pi(h)}$ and $B_t = \sum_{h=1}^{t} b_{\pi(h)}$; the objective value is that of (2),

$$WC_{\max}(\pi) = \max_{1 \le t \le n} w_{\pi(t)}\, C_{\pi(t)}.$$

The fitness of a schedule is the criterion itself, $WC_{\max}(\pi)$, and no normalised scale is fixed for it. Figure 4 shows the encoding and the decoding on the six-job instance of Example 4.1.

5.2.2 Initial population

Random starting populations waste most of the search effort. The initial population P_0 of size N is therefore built from Johnson's rule (Johnson, 1954). Its first individual is the Johnson schedule π^{J} itself, which minimizes the makespan and so shortens every completion time at once. The remaining $N-1$ individuals are neighbours of that schedule, each of them a copy of it with a few neighbouring jobs exchanged, the number of exchanges being drawn uniformly between one and $\lfloor n/4 \rfloor$. A uniformly random permutation carries nothing of the instance, while a neighbour of Johnson's order is already short on the makespan. The rule reads the two processing times alone and knows nothing of the weights, so the population starts from the order that is shortest on the machines and the criterion is brought in by the block move that follows, one weight at a time. The population size grows with the instance and is capped, $N = \min(20, \max(6, \lceil n/2 \rceil))$: a small instance is given a small population, and a large one keeps the same twenty individuals, which are enough for the block mutation and the local search to reach what a population as large as the instance reaches. A generation is dominated by the mutation, which evaluates n positions of a block at $O(n)$ time each, and therefore costs $O(\lambda n^2)$, where λ is the number of offspring the generation produces.

5.2.3 Selection, crossover and survival

A generation turns the population into its successor by three moves. The parents are selected, the parents are recombined into offspring, and the fittest of parents and offspring survive. The algorithm runs G generations of them, and each of them produces λ offspring from its N parents, so that the operators of one generation are run several times over the same parents. The population it starts from carries nothing of the weights, so the generations are spent building the criterion into the schedule rather than moving an order that already has it, and both G and λ rise with the instance: the depth a run reaches grows with the number of generations, since the elite of one is a parent of the next, and the number of classes an instance has to be sorted into does not. A generation costs the evaluation of the $N + \lambda$ schedules it holds in $O(n)$ time each and one pass of the operators over the offspring, whose most expensive move is the mutation of a child in $O(n^2)$, and the search therefore costs $O(G\lambda n^2)$. What is left of it after the last generation is the local search that closes the algorithm.

Selection. Binary tournament selection forms the mating pool M. In each round, two individuals are drawn uniformly at random from P_{g-1}. The better of the two on the Pareto rank wins and is copied into the mating pool M, ties being broken by the crowding distance. Both individuals are returned to the population, so that a parent may be drawn more than once, and the round is repeated until M_g contains N individuals. Tournament selection is preferred over roulette-wheel selection because it compares two individuals directly rather than through a probability proportional to a fitness value. It therefore remains selective even as the population converges and the schedules of near-optimal individuals become very close.

Crossover. The operator recombines two parent schedules into offspring. These inherit both the relative order and the absolute positions of jobs, the two ingredients a permutation schedule is built from. We use the partially mapped crossover (PMX), the standard operator for permutation-encoded genetic algorithms. Two cut points $p_1 < p_2$ are drawn uniformly at random, and the middle segment $\pi_1[p_1..p_2]$ is copied verbatim into the child. The remaining positions are filled from the second parent π_2. A gene $\pi_2[t]$ of such a position $t \notin [p_1, p_2]$ clashes with the copy only when the copied segment already holds it. The outer positions receive pairwise distinct genes of π_2, so no other clash is possible, and a job cannot be written twice. Such a gene is replaced through the mapping that the segment induces between the two parents, $j \leftarrow \pi_2[\pi_1^{-1}(j)]$. The job π_1 holds at a segment position then gives way to the job π_2 holds at the same position. The replacement is repeated while the gene it yields already occurs in the child. The mapping is injective, so the chain it follows visits distinct jobs and stops after at most $p_2 - p_1 + 1$ replacements. Every position then receives exactly one job, and the child is always a valid permutation. PMX transfers contiguous blocks of jobs, which realize the front-heavy structure identified in Section 4, while still recombining the two parents' orders on the outer positions. Swapping the roles of π_1 and π_2 yields the second

child, so that one application replaces the two parents by two offspring. Figure 5 shows both children of one application. Its two upper rows are the parents and its two lower rows the offspring, the grey cells are the middle segment the child copies (π' takes the segment of π_1, π'' that of π_2), and the three double arrows between the parents are the mapping the segment induces. The red cells, numbered in the order in which they are visited, are those of the clash at position 1 of π': the gene taken there, $\pi_2[1] = J_6$, is one the segment already holds, at position 3, and the chain $J_6 \to J_1 \to J_5$ stops at the free job J_5, which is the value the position finally receives.

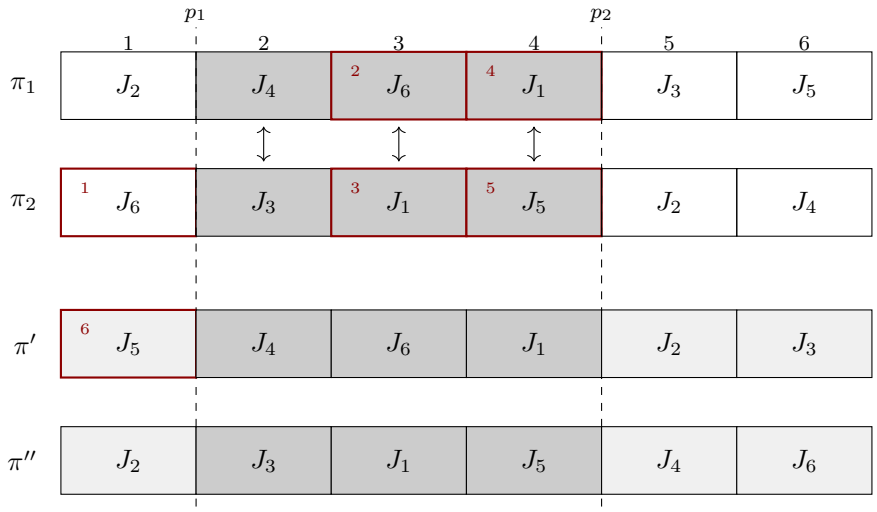


Figure 5: One application of the partially mapped crossover, with $p_1 = 2$ and $p_2 = 4$

Algorithm 6: Partially Mapped Crossover (PMX) for $F2 \parallel WC_{\max}$

Input: Parents π_1, π_2; cut points $p_1 < p_2$.

Step 1. [Copy segment] Set $\pi'[t] \leftarrow \pi_1[t]$ for $t = p_1, \ldots, p_2$.
Step 2. [Fill outer positions] **for** $t = 1$ **to** n, $t \notin [p_1, p_2]$ **do**
  $j \leftarrow \pi_2[t]$
  while j *occurs in* π' **do**
    $j \leftarrow \pi_2[\pi_1^{-1}(j)]$ `// follow the PMX mapping`
  end
  $\pi'[t] \leftarrow j$
end

Output: Child π'.

Survival. The parents of P_{g-1} and the offspring O_g are pooled, so that a good incumbent is never discarded. The N survivors of $P_{g-1} \cup O_g$ are chosen by the non-dominated sorting and the crowding distance of NSGA-II (Deb et al., 2002). Those rank the pooled individuals on three quantities, the makespan $C_{\max}(\pi)$ of the schedule, the weight $w_{\pi(n)}$ of the job that closes it, and the criterion $WC_{\max}(\pi)$ itself. Whole Pareto fronts are kept while they fit into

the budget of N individuals. The front that overflows it is cut by the crowding distance, its least crowded individuals first. The first two carry the criterion: the weighted completion of the closing job is exactly their product,

$$w_{\pi(n)}\, C_{\max}(\pi) \leq WC_{\max}(\pi),$$

so ranking on them keeps both its ingredients in view, a schedule that ends early and one whose last job is light, and keeps the weights in the account, which a search that pressed the makespan alone would leave out. They are not the criterion, however, since a schedule can end early and light and still be a poor one, and a search that ranked them alone would spend its selection on the trade-off between them rather than on what the method is measured on. The criterion is therefore ranked with them rather than tracked beside them. It is also tracked on its own, over every child of every generation, and the best schedule seen on it is carried into the next generation: where the ranking does not keep it, it takes the place of the individual the ranking leaves last, so that an improvement is a parent of the generation that follows and not merely a value that was recorded once.

5.2.4 Mutation

The crossover reads nothing of the weights, so the mutation is what keeps the search moving across the weights: it moves whole weights rather than single jobs, and it is what carries the population across the block orders the criterion rewards.

The move is the one the block structure of Lemma 4.1 suggests. An optimal schedule holds one block per distinct weight, the blocks in non-increasing order of it, and the arrangement the search has to reach is therefore an order of the blocks rather than of the jobs. A single job moved at a time cannot reach it quickly: placing one job leaves the other jobs of its weight where they were, so the search would need as many accepted moves as there are jobs. The mutation therefore moves a block. The jobs of one weight leave the schedule, keeping the order they hold in it, and the run is put back at the position that minimizes $WC_{\max}$ over the whole schedule, which is evaluated by (1) at every one of the positions the run can take. The weight is the one of a job attaining $WC_{\max}(\pi)$, since that is the job that closes the criterion, and with probability $1/2$ a weight drawn at random is used instead, so that the order of the blocks is explored and not only repaired.

The move is a variation and not an improvement. Moving a block forward delays every job it passes, so the criterion may rise while the block order moves towards the one Lemma 4.1 prescribes, and there are orders the criterion rewards that no sequence of accepted moves reaches. The move is therefore proposed whether or not it lowers the criterion, and it is the survival of the generation, and the elite carried through it, that decide what becomes of it.

The complete algorithm The four parts above assemble into Algorithm 8, which runs as many generations as the instance is given: the population it starts from is short on the

Algorithm 7: Block Mutation for $F2 \parallel WC_{\max}$

Input: Schedule π; weights w_j.

Step 1. [Block] Evaluate $WC_{\max}(\pi)$ by (1) and let j^* be a job attaining the maximum; with probability $1/2$ let $W \leftarrow w_{j^*}$, otherwise let W be a weight drawn at random from the instance.

Step 2. [Move] Form the run of the jobs of weight W, keeping the order π holds them in, and remove it from π.

for *every position r of the remaining schedule* **do**

  Form $\pi^{(r)}$ by inserting the run at position r and evaluate $WC_{\max}(\pi^{(r)})$.

end

$\pi \leftarrow \pi^{(r^*)}$, where $r^* = \arg\min_r WC_{\max}(\pi^{(r)})$, whichever way the criterion moves.

Output: Schedule π.

Algorithm 8: Genetic Algorithm for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$; population size N; generations G; offspring λ; probabilities p_c, p_m; reinsertion count k.

Step 1. [Initialization] Build P_0 from Johnson's schedule and $N-1$ neighbours of it; evaluate all individuals; set π^* to the best.

Step 2. [Evolution] **for** $g = 1$ **to** G **do**

  Selection Form the mating pool M_g from P_{g-1} by binary tournament selection.

  Crossover while $|O_g| < \lambda$ **do**

    Draw two parents from M_g; with probability p_c apply Algorithm 6 to obtain two children, otherwise copy the parents into the offspring set O_g.

  end

  Mutation Apply Algorithm 7 to each offspring with probability p_m; evaluate every child by (1).

  Survival $P_g \leftarrow$ the N survivors of $P_{g-1} \cup O_g$, taken by non-dominated sorting on the three objectives and, on the last front, by crowding distance.

  Elite If a child of O_g improves on $WC_{\max}(\pi^*)$, replace π^* by it, and if π^* is not among the survivors, let it take the place of the one the ranking leaves last, so that it is a parent of generation $g+1$.

end

Step 3. [Local search] **repeat**

  Let $J_1, \ldots, J_k$ be the jobs of π^* with the largest $w_j C_j$.

  Form, for every J_i and every position r, the schedule obtained by moving J_i to position r, and evaluate it by (1); let π' be the best of them.

  if $WC_{\max}(\pi') < WC_{\max}(\pi^*)$ **then**

    $\pi^* \leftarrow \pi'$

  end

until *no reinsertion of the step improves*

Output: Best schedule π^*.

makespan but carries nothing of the weights, so the criterion has to be iterated into the schedule by the mutation before a good order appears. Its closing step is a local search on the schedule the population has reached, over the k jobs that carry the largest weighted completion times, which are reinserted, each at the position that lowers the criterion most, the pass being repeated while it keeps lowering it. Those jobs are the ones that decide the criterion, so reinserting them is where an insertion can still pay. The descent stops as soon as none of them improves.

6 Experiments

All methods were coded in Python 3.13 and run on a machine with an Intel Core Ultra 5 230F processor and 16 GB of memory, under Windows 11. The MILP model was solved with Gurobi 13.0 on a single thread, so that the reported times are comparable across configurations.

6.1 Experimental setup

Every method is run once on every instance and given the same budget, and all of them evaluate the objective by the same recurrence (1), so that the comparison holds the objective fixed and varies only the search. Two readings are taken. The dynamic program, the only exact method of the study, is read on the number of instances whose optimum it establishes and on its running time. Every other method is read on its relative percentage gap

$$\mathrm{gap}_A = \frac{z_A - z_{\mathrm{base}}}{z_{\mathrm{base}}}, \tag{31}$$

where z_A is the objective value of the schedule it returned and z_{base} the reference value of the instance, and on the CPU time it requires; the gap is reported in per cent and the time in milliseconds, each by its mean and its maximum over the instances of the configuration. On the small scale the reference is the optimum, which is the value the model returns, so that the model is read on a gap of zero and every heuristic on its distance from the optimum. On the large scale, where no exact method is within reach, the reference is the lower bound LB of (28) – the largest of the three thresholds LB_1, LB_2 and LB_3 of Section 4.3, each of which the instance determines on its own – so that no schedule of a method enters the denominator of its gap.

The parameters of the genetic algorithm were fixed by a calibration run before the study, and each is a function of the instance size, so that one rule covers every size of both scales. Its population is $N = \min(20, \max(6, \lceil n/2 \rceil))$ individuals, $6, 8, 10, \ldots, 20$ over the sizes of the small scale and twenty from $n = 40$ on. Both of its probabilities rise with the instance, since the more jobs it holds the more of the search has to be spent moving them rather than preserving them: the crossover probability is $p_c = \min(1,\ 19/20 + n/4000)$, from 0.95 at

$n = 10$ to one at $n = 200$, and the mutation probability is $p_m = \min(1,\ 1/2 + n/500)$, from 0.52 at $n = 10$ to one from $n = 250$ on. The two rates say which operator is expected to do the work: the crossover is applied to almost every pair of parents over the whole study and has little room to rise, while the mutation, which carries the population across the weights, is given a share that doubles. A small instance is therefore given a small population and mutated sparingly, and a large one the full budget.

Three settings the scales do not share. The first is the number of generations, which is one round of the operators and therefore the depth to which the elite is refined: $G = \max(2, \lfloor n/4 \rfloor)$ on the small scale, from 2 at $n = 10$ to 15 at $n = 60$, and $G = \max(16, \lfloor n^2/2000 \rfloor)$ on the large one, from 16 at $n = 100$ to 80 at $n = 400$ – a floor and a quadratic growth, since the depth may be neither too small at the bottom of the scale nor growing more slowly than the instance. The second is the number of offspring a generation produces, $\lambda = \max(8, \lfloor n^2/12 \rfloor)$ on the small scale and $\lambda = \min(200, 60 + \lfloor n/2 \rfloor)$ on the large one, $8, 18, \ldots, 300$ over the first and 110 at $n = 100$ up to the cap of 200 from $n = 280$ over the second: it sets how many times the operators are run over the parents in one generation, so that a generation offers the offspring of its parents λ/N chances rather than the single one a child per parent would give it. The third is the width of the local search of Algorithm 8: the small scale reinserts $k = \max(1, \lfloor n/8 \rfloor)$ jobs, one job in eight, which is 1, 2 and 7 at $n = 10$, 20 and 60, and the large one $k = \max(2, \lfloor 12n/100 \rfloor - 10)$ jobs, twelve in a hundred less ten, which is 2, 14 and 38 at $n = 100$, 200 and 400; the width of the large scale therefore grows faster than the instance itself, one job in fifty at a hundred and one in ten at four hundred.

The initial population is built from Johnson's rule on both scales. That rule reads the two processing times alone, so the criterion has to be built into the schedule by the operators, and the three settings above say how much of the search is spent on that. The two scales are therefore read for different things: the small one for the course of the search, whose settings are the narrow ones and whose gap against the optimum falls from 17 per cent at ten jobs to 4 at sixty, closing on the construction rule without passing it; the large one for what the search reaches when it is given room, its settings growing faster than the instance and its gap closing as the instance grows.

The instances were generated in three groups, and every integer of an instance is drawn from a normal distribution on the integers of its range: the mean at the middle of the range, the standard deviation a sixth of its width, so that the two ends lie three standard deviations away, and the draw rounded to an integer and clamped to the range. The first group, on which the dynamic program is reported, varies the instance size: 40, 60, 80 and 100, crossed with the weight count $K = 2$, whose weights are the geometric ladder $1, 10$, and with the two ranges $[1, 40]$ and $[1, 80]$ that the other groups use as well. Its ladder rises by 20 a step and stops at the largest size the time limit still reaches: at $[1, 40]$ every instance is settled well within the limit, while on $[1, 80]$ the limit binds at a hundred jobs and cuts six of the twenty

short. This group is not the set on which the methods are compared. The running time $O(nKV^{3K-3})$ of the dynamic program grows so fast in the number K of distinct weights that the sizes the other algorithms are read at are out of its reach, so it is reported on a set of its own.

Table 1: The dynamic program, $K = 2$

n	$a_j, b_j \in [1, 40]$			$a_j, b_j \in [1, 80]$		
	$WC_{\max}$	time (ms)	opt.	$WC_{\max}$	time (ms)	opt.
40	4335.0 5480.0	1465.55 4257.81	20/20	8604.5 10590.0	4984.88 24448.80	20/20
60	6340.0 7270.0	14233.26 31363.33	20/20	12344.5 15020.0	52541.32 113542.29	20/20
80	8221.0 9750.0	45134.23 109544.47	20/20	17341.5 19580.0	134034.27 278535.25	20/20
100	10647.0 12350.0	89585.46 168850.77	20/20	21544.3 24240.0	329010.01 628500.13	14/20

The two groups that follow are the comparison, and both carry the two ranges of the first, so that the width of the range is a factor every group shares. An instance of theirs carries at most thirty distinct weights, drawn by the same rule on $\{1, \ldots, 30\}$: the number of its weight classes is arbitrary below that bound, the populous classes are the ones near the middle, and neighbouring integers replace the decades of the first group. Its processing times are drawn the same way, and independently for every job, on one of $[1, 40]$ and $[1, 80]$, which are read as a factor and named in the caption of every table below. The small group varies the instance size over $10, 15, \ldots, 60$ and the large one over $100, 120, \ldots, 400$: the model is compared with the heuristics at the small sizes, where it is exact and settles them, and the heuristics alone are reported at the large ones, where no exact method is within reach and the gap of each is measured against LB of (28). Between them the three groups carry the three factors of the study: the instance size, the number of distinct weights and the width of the processing-time range. Every configuration is replicated over twenty independent instances and every figure below is the average of them; every method is given the same time limit of 600 s, and a method that reaches it is recorded with the value it has attained rather than omitted. Every table below is read the same way. A row is an instance size, the columns are grouped by method and, in the table of the dynamic program, by processing-time range, and an entry is the pair (mean, maximum) of the quantity its column names over the twenty instances of the configuration, the mean above and the maximum below it in the smaller type. Gaps are in per cent and running times in milliseconds, and the "opt." column of the dynamic program gives the number of instances whose optimum it establishes.

The model of Section 2 and the two heuristics of Section 5 were applied to every instance of the small group, and the heuristics alone to every instance of the large one, where no exact method is within reach; the dynamic program of Algorithm 2 was applied to the first group alone, the only one its running time reaches.

Each table is read for one question. Table 1 reports the dynamic program on its own instances, and is read for how far the exact method reaches and what it costs to get there; Tables 2 to 3 report the small scale, one table per range, and are read for the gap and the running time of every method as the instance grows; Tables 4 to 5 report the large scale, one table per range, and are read for the same two quantities under the instance size. Read together, they report a change of leader rather than a fixed order: which method comes closest to the reference depends on the scale, and on the size within a scale.

6.2 Results

The model settles every instance of the small scale to optimality – 20 of 20 in each of its 22 configurations, 440 instances in all – so the gaps reported against it are distances to an optimum and not to the incumbent of a run that ran out of time. What grows is its cost: the mean time of an instance rises from 50.6 ms at ten jobs to 80.1 s at sixty on $[1, 40]$, and from 40.2 ms to 43.5 s on $[1, 80]$, while the value it returns rises with the range as well – the mean optimum of a ten-job instance is 2510.7 on $[1, 40]$ and 5433.0 on $[1, 80]$, and that of a sixty-job instance 12323.6 and 25266.1. The slowest instance of the scale takes 299.6 s of the 600 s a run is allowed, so the exact side is complete here but not roomy: the limit is close by the top of the scale.

The dynamic program is the only method of the study that certifies an optimum, and its strength is that it is exact wherever it returns: seven of the eight configurations settle all twenty of their instances, and a forty-job instance costs 1.47 s on $[1, 40]$ and 4.98 s on $[1, 80]$. Its weakness is reach, and it is bounded from two sides. The first is the number of weight classes. Its state space is bounded by $2^K V^{3K-3}$, with V the total processing time of the instance (Section 4), so that a third class multiplies the number of states by $2V^3$: at $K = 3$ the memory of the machine the runs were made on is exhausted and the program does not return, which is why the group is reported at $K = 2$ alone. The second is the instance size and the width of the range, which enter through V as well. At $a_j, b_j \in [1, 40]$ the mean running time rises from 1.47 s at forty jobs to 14.23 s at sixty, 45.13 s at eighty and 89.59 s at a hundred – a factor of 61 over the ladder, against a criterion that grows only from 4335.0 to 10647.0, a factor of 2.5. The range multiplies that time by between 3.0 and 3.7 at every size, the two extremes being 1.47 against 4.98 s at forty jobs and 89.59 against 329.01 s at a hundred. The two bounds meet at the top of the ladder: at a hundred jobs on $[1, 80]$ the 600 s limit cuts six of the twenty instances short, so that the configuration settles 14/20, and over the fourteen it settles the mean time is 329.01 s with a largest of 628.50 s, past the limit because the deadline is checked once per job and not inside one. The exact method is therefore cheap at forty jobs and out of reach beyond a hundred, while the heuristics are read to four hundred.

The two rules that read an instance without searching it are close to the optimum and

far apart in what they cost. The 2-approximation, which reads the weights alone, has a mean gap of 3.52 per cent over the whole small scale – 3.47 on $[1, 40]$ and 3.56 on $[1, 80]$ – and reaches the optimum on 36 of the 440 instances, its worst 16.71 per cent away. NEH has a mean gap of 2.53 per cent, 2.39 and 2.67 on the two ranges, reaches the optimum on 99 of the 440, and its worst instance is 11.27 per cent away. The two reverse over the scale: NEH is the closer at nine of the eleven sizes on $[1, 40]$ and at seven on $[1, 80]$, and the 2-approximation takes the lead at the largest sizes, where the order of the weights is what the criterion rewards. Their costs are not of the same order either: 0.02 ms an instance on average for the 2-approximation, never above 0.05 ms, against 2.68 ms for NEH, which grows with the instance to 8.35 ms at sixty jobs; the model is a thousand times above both, at 13.85 s an instance on the average of the scale.

The genetic algorithm is the only method whose gap closes over the small scale, and that is its strength here: it approaches the optimum from far away. Its mean gap falls from 16.83 to 3.92 per cent on $[1, 40]$ and from 13.16 to 3.24 per cent on $[1, 80]$, which leaves it within a point of both rules at sixty jobs. What the rest of the scale shows is what that approach costs. It starts as the worst of the three, at three to twenty times the gap of the rules at ten jobs; its course is not monotone, rising to 18.96 and 18.97 per cent at fifteen jobs and staying above 14 per cent until thirty-five, where the instances are harder for it by more than the growth of its own budget makes up for; and over the 22 configurations it stands at 10.75 per cent, against 3.52 for the 2-approximation and 2.53 for NEH, with a worst instance 67.41 per cent from the optimum, at ten jobs. Its cost is above both rules by three orders of magnitude: 0.55 ms an instance at ten jobs and 1.65 s at sixty, against the 80.1 s the model needs at that size. On this scale the search neither beats the rules nor approaches them cheaply; what it shows is the direction in which it moves, and that is read on the large scale.

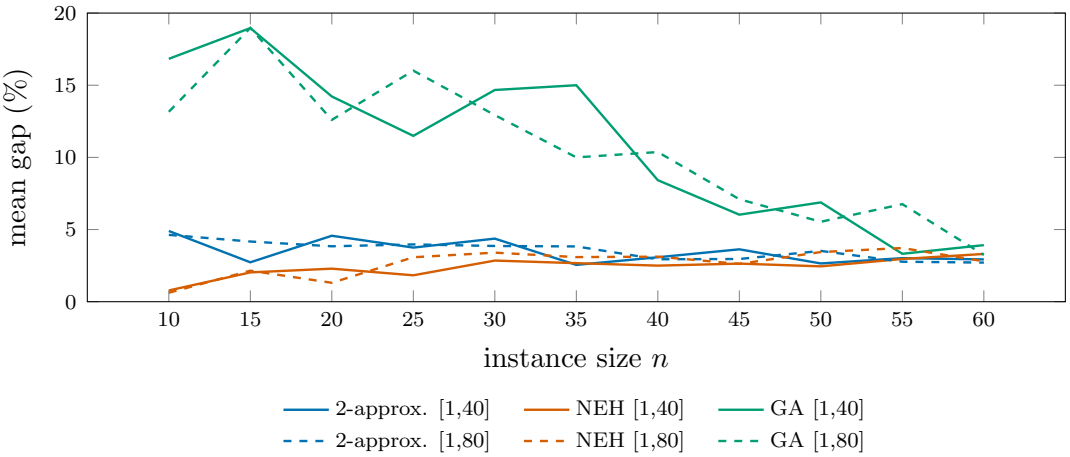


Figure 6: Mean relative gap of the three heuristics on the small scale, against the optimum the model returned. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

On the large scale the three methods separate for good. The rules are flat: the 2-approximation is the closest on average at 1.81 per cent over the 32 configurations, its mean

Table 2: The small scale, $a_j, b_j \in [1, 40]$: MILP, 2-approx., NEH, GA

n	MILP		2-approx.		NEH		GA	
	$WC_{\max}$	time (ms)	gap	time (ms)	gap	time (ms)	gap	time (ms)
	2510.7	50.61	4.90	0.01	**0.78**	0.08	16.83	0.55
10	3192.0	80.45	14.18	0.01	8.68	0.27	67.41	0.75
	3682.3	134.97	2.73	0.01	**2.03**	0.18	18.96	2.48
15	4746.0	275.95	8.70	0.02	7.97	0.22	42.04	2.71
	4742.4	377.45	4.57	0.01	**2.29**	0.39	14.22	10.27
20	5759.0	886.36	9.48	0.02	9.71	0.42	34.19	11.21
	5736.1	725.39	3.75	0.01	**1.83**	0.69	11.49	26.07
25	7260.0	2338.97	10.79	0.02	9.19	0.73	27.85	28.68
	6513.6	1269.43	4.37	0.01	**2.85**	1.14	14.67	59.60
30	8892.0	2396.07	9.34	0.03	9.31	1.20	43.61	64.39
	7836.4	2973.48	**2.55**	0.02	2.67	1.74	15.00	117.27
35	8931.0	10776.15	9.68	0.03	7.67	1.79	30.68	126.27
	8760.2	5106.22	3.08	0.02	**2.50**	2.53	8.42	240.93
40	10153.0	15912.26	7.69	0.03	8.71	2.61	21.17	262.61
	9564.3	12793.08	3.63	0.02	**2.64**	3.52	6.03	403.04
45	11076.0	37697.87	7.98	0.03	7.12	3.66	17.02	443.34
	10631.6	19520.52	2.65	0.02	**2.45**	4.92	6.88	659.74
50	11774.0	61667.95	7.24	0.03	7.10	5.37	21.46	732.34
	11760.6	36700.15	3.02	0.02	**2.96**	6.29	3.32	1021.85
55	12992.0	176554.12	6.35	0.05	8.08	6.50	10.77	1099.12
	12323.6	80106.40	**2.93**	0.02	3.31	8.05	3.92	1652.00
60	14703.0	299585.38	8.87	0.04	6.72	8.35	10.07	1721.89

Table 3: The small scale, $a_j, b_j \in [1, 80]$: MILP, 2-approx., NEH, GA

n	MILP		2-approx.		NEH		GA	
	$WC_{\max}$	time (ms)	gap	time (ms)	gap	time (ms)	gap	time (ms)
	5433.0	40.18	4.63	0.01	**0.62**	0.08	13.16	0.53
10	7365.0	86.55	16.71	0.01	2.07	0.16	41.92	0.58
	6941.7	163.01	4.17	0.01	**2.15**	0.18	18.97	2.32
15	9035.0	299.74	11.66	0.02	8.18	0.20	37.17	2.69
	9029.8	386.24	3.84	0.01	**1.31**	0.39	12.60	10.02
20	10906.0	653.07	13.68	0.02	7.42	0.49	37.31	11.10
	10971.5	692.05	3.97	0.01	**3.08**	0.71	16.01	26.02
25	13013.0	1391.04	11.14	0.03	10.71	0.75	27.13	28.33
	12832.5	1869.80	3.86	0.01	**3.40**	1.14	12.93	57.85
30	15232.0	9374.60	8.81	0.02	7.90	1.17	30.56	65.72
	15171.0	3264.38	3.83	0.02	**3.09**	1.75	10.00	115.93
35	16940.0	10566.47	7.84	0.02	11.27	1.82	19.84	124.35
	16912.4	8283.81	**2.94**	0.02	3.12	2.54	10.37	237.39
40	19513.0	25306.71	11.28	0.03	11.03	2.66	29.73	255.75
	19067.3	9174.69	2.96	0.02	**2.62**	3.55	7.10	405.21
45	23062.0	18458.57	7.32	0.04	7.82	4.00	23.50	435.56
	20795.4	22314.68	3.52	0.02	**3.44**	4.78	5.54	657.49
50	23505.0	51257.77	9.63	0.03	9.71	4.94	14.98	705.72
	22517.8	55263.13	**2.77**	0.02	3.72	6.23	6.76	1021.42
55	27573.0	140840.34	9.26	0.05	10.07	6.42	17.55	1151.79
	25266.1	43535.09	**2.71**	0.03	2.80	7.97	3.24	1652.52
60	29614.0	103537.34	8.44	0.04	6.90	8.18	8.86	1825.83

never above 3.20 per cent and its worst instance 7.57 per cent away, and NEH is the furthest at 4.40 per cent, its gap hardly moving over the scale – 5.02 at a hundred jobs against 3.60 at four hundred on $[1, 40]$, and 5.39 against 4.47 on $[1, 80]$, with a worst instance 12.64 per cent away. The genetic algorithm is the only method whose gap falls as the instance grows, and it falls steeply: 10.88 per cent at a hundred and eighty jobs, 5.49 at two hundred, 2.09 at

two hundred and forty and 0.22 at four hundred on $[1, 40]$, and 6.99, 6.34, 2.09 and 0.13 per cent on $[1, 80]$. It passes NEH at 240 jobs and the 2-approximation at about 260 on $[1, 80]$ and 280 on $[1, 40]$, and over the 640 instances of the scale it attains the best value on 318 of them, against 283 for the 2-approximation and 42 for NEH. What it costs is time and the bottom of the scale: 70.8 s an instance on average, against 0.076 ms for the 2-approximation and 0.76 s for NEH, with 313.5 s on its slowest instance of the 600 s a run allows, and its worst configuration and its worst instance both at the bottom, at 10.88 and 28.17 per cent.

The two ranges agree on the ordering of the two rules and disagree on the search. The 2-approximation is closer on the wider range, 1.70 per cent against 1.92, and NEH further from it, 4.79 against 4.02, so that the 2-approximation leads NEH on both ranges while the search overtakes it on one: 4.28 against 4.02 per cent on $[1, 40]$, where the two are within a quarter of a point of one another, and 3.65 against 4.79 on $[1, 80]$.

Table 4: The large scale, $a_j, b_j \in [1, 40]$: 2-approx., NEH, GA

n	2-approx.		NEH		GA	
	gap	time (ms)	gap	time (ms)	gap	time (ms)
100	**3.20** 7.57	0.03 0.07	5.02 7.69	35.26 36.47	10.58 28.17	1384.37 1439.61
120	**2.87** 5.08	0.03 0.05	4.26 11.00	61.35 67.38	9.71 21.40	2203.41 2271.48
140	**2.69** 6.12	0.03 0.04	4.08 6.16	92.04 97.28	10.76 23.19	3180.44 3283.81
160	**1.89** 3.70	0.04 0.04	4.59 7.14	134.46 137.52	9.58 20.50	4555.65 4747.43
180	**2.35** 5.34	0.04 0.07	4.11 7.33	191.30 198.87	10.88 20.41	6435.09 6629.00
200	**2.06** 5.11	0.05 0.06	3.89 8.45	260.86 266.55	5.49 14.18	10931.38 11172.88
220	**2.00** 5.12	0.06 0.10	3.90 5.88	344.81 353.78	4.33 10.78	17399.07 17848.49
240	**1.65** 3.54	0.06 0.10	4.21 7.15	445.75 455.58	2.09 8.10	26442.75 27195.43
260	**1.67** 3.88	0.07 0.08	4.02 6.73	568.66 576.57	1.73 5.95	39325.56 40106.05
280	1.63 4.59	0.07 0.08	3.97 7.33	720.96 737.22	**0.81** 3.79	57650.47 58582.96
300	2.05 5.43	0.08 0.11	4.11 6.21	892.01 904.59	**0.68** 1.97	77091.75 79319.03
320	1.35 2.71	0.09 0.14	3.59 6.12	1090.79 1111.77	**0.59** 2.36	99565.36 101622.73
340	1.31 4.02	0.10 0.15	3.54 5.47	1313.39 1325.81	**0.43** 1.94	126205.10 127684.57
360	1.16 2.85	0.11 0.14	3.67 7.22	1562.68 1583.38	**0.19** 0.94	158777.05 160802.00
380	1.31 3.08	0.12 0.15	3.79 5.99	1935.62 2143.48	**0.37** 1.59	205483.27 219431.17
400	1.51 2.99	0.20 0.89	3.60 4.93	2405.22 2877.28	**0.22** 1.06	276234.16 331651.06

The two scales are read for different things, and their tables show why. On the small scale the three methods end within a point of one another, the search having travelled the whole way to join the two rules; on the large one the rules stay within a few points of the bound at every size while the search falls through both, and it is there, and only there, that

Table 5: The large scale, $a_j, b_j \in [1, 80]$: 2-approx., NEH, GA

n	2-approx.		NEH		GA	
	gap	time (ms)	gap	time (ms)	gap	time (ms)
100	**2.75** 5.26	0.03 0.05	5.39 12.64	35.66 37.43	8.95 23.55	1409.90 1496.96
120	**2.44** 4.66	0.03 0.06	4.64 6.47	60.32 66.07	9.06 20.88	2167.31 2249.25
140	**2.15** 6.02	0.03 0.04	4.81 8.43	90.89 93.33	7.24 15.10	3136.94 3209.90
160	**2.60** 5.49	0.04 0.04	3.97 7.54	134.96 139.10	10.21 21.92	4571.04 4726.96
180	**2.14** 4.70	0.04 0.05	4.66 7.17	190.18 195.30	6.99 17.08	6429.74 6726.51
200	**1.87** 6.60	0.05 0.07	5.00 8.60	260.31 265.96	6.34 17.48	10916.49 11382.00
220	**1.37** 2.98	0.05 0.06	5.11 7.64	342.95 347.44	2.47 7.33	17326.84 17825.02
240	**1.26** 2.43	0.06 0.07	4.64 8.01	445.17 460.79	2.09 6.77	26299.84 27028.61
260	1.87 4.87	0.07 0.08	5.59 8.02	569.50 582.71	**1.85** 5.73	39464.28 40446.42
280	1.41 3.11	0.07 0.09	3.90 6.46	720.36 731.04	**0.74** 2.42	57760.02 58914.31
300	1.20 2.92	0.09 0.13	4.90 6.68	891.43 907.87	**1.09** 3.79	76949.10 78040.10
320	1.23 3.09	0.09 0.12	4.30 6.03	1092.38 1114.11	**0.51** 1.78	99498.39 100647.08
340	1.05 3.00	0.10 0.12	4.63 6.59	1312.40 1329.73	**0.20** 1.02	125947.63 126830.87
360	1.17 2.17	0.11 0.13	5.37 6.78	1586.33 1648.92	**0.24** 1.43	160224.94 164703.79
380	1.60 3.28	0.12 0.13	5.21 7.25	1935.08 1975.71	**0.36** 1.80	208505.05 212639.22
400	1.11 3.71	0.20 0.43	4.47 7.66	2729.24 3169.58	**0.13** 0.65	313465.36 350424.29

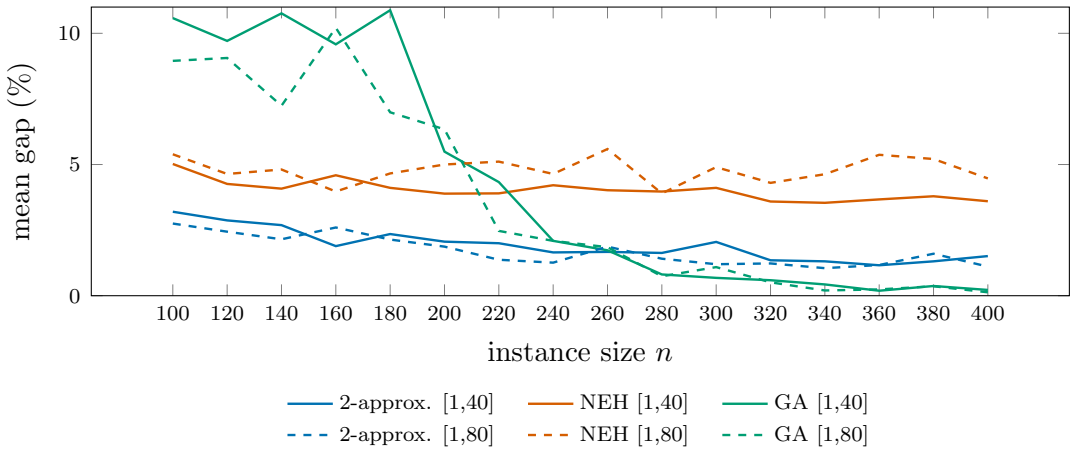


Figure 7: Mean relative gap of the three heuristics on the large scale, against the lower bound LB. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

the population buys a value the rules do not reach.

6.3 Discussion

What the study reads, then, is a division of labour rather than a ranking. The 2-approximation is the cheapest method by six orders of magnitude and stays within five per cent of the optimum on the small scale and within 1.81 per cent of LB on the large one, which says that the factor of two of Theorem 4.2 is loose by an order of magnitude on instances of this kind. What it cannot do is read anything but the weights, and that is the ground NEH takes: NEH reads the two processing times and no weight, which makes it the closest method at the bottom of the small scale and the furthest at the top of the large one, where its gap does not close as the instance grows.

The two methods that carry the study beyond a rule sit at opposite ends of it. The dynamic program is the only one that certifies an optimum, and it does so cheaply where it applies – 1.47 s for forty jobs on $[1, 40]$ – but its applicability is the restriction: two weight classes, at most a hundred jobs, and a third class that exhausts the memory of the machine before the search starts. The genetic algorithm is the reverse. It certifies nothing, it costs three orders of magnitude more than the rules on the small scale and more than ninety times NEH on the large one, and it is nevertheless the only method that beats both rules, on 318 of the 640 large instances and from about 260 jobs on. Neither is read on the instances of the other – the dynamic program on a set of its own at $K = 2$, the search on instances that carry up to thirty weight classes – so what separates them is not a common benchmark but the question each answers: whether the optimum can be certified at all, or whether a schedule that beats every rule can be built at the sizes that arise. The reason the search needs the room is the start it is given: Johnson's rule carries no weight, so its early generations are spent building the criterion into the schedule before it can profit from it.

Two properties of the comparison are worth keeping in view. The large-scale gaps are distances to LB, which caps the optimum by the three prefix bounds of (28) and is tight only where the heaviest jobs are also the earliest ones; what they order reliably is therefore the methods and not their absolute quality. And the answers depend on the processing-time range: the average advantage of the search over NEH, 3.97 against 4.40 per cent, is earned on $[1, 80]$ alone.

What would extend the study follows from each of its limits. On the exact side the dynamic program is stopped by memory, so dominance rules that drop a state another state beats, or a solution of one weight class at a time, are the way past $K = 2$, with the scheme of Section 4.4 the memory-lean alternative where they are not enough. On the side of the search, seeding the population with the order the 2-approximation returns costs microseconds and carries the weights the Johnson order lacks, which is the cheap way to move the size at which the search overtakes the rules. Both point at the same economy, of spending the budget where the criterion still moves, which the calibration of the large scale measured when it found the count of generations to be the lever of that scale and the width of the

insertion descent nearly inert.

7 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.

Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.

Garey, M. R., Johnson, D. S., and Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2):117–129.

Guo, L., Xia, F., and Zhang, X. (2026). Single machine scheduling to minimize maximum earliness/tardiness cost with job rejection. *CCF Transactions on High Performance Computing*, 8(3):314–330.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Lawler, E. L. (1973). Optimal sequencing of a single machine subject to precedence constraints. *Management Science*, 19(5):544–546.

Li, W. (2015). Best-possible online algorithms for single machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 32(4):1550030.

Lu, L., Zuo, L., Zhang, L., and Ou, J. (2025). Order acceptance and scheduling with weighted makespan. *4OR*, 23(3):247–267.

Nawaz, M., Enscore, E. E., and Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95.

Panwalkar, S. S. and Koulamas, C. (2015). On equivalence between the proportionate flow shop and single-machine scheduling problems. *Naval Research Logistics*, 62(7):595–603.

Panwalkar, S. S., Smith, M. L., and Koulamas, C. (2013). Review of the ordered and proportionate flow shop scheduling research. *Naval Research Logistics*, 60(1):46–55.

Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, 6 edition.

Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vn-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.

Qiao, Y., Wu, N., He, Y., Li, Z., and Chen, T. (2022). Adaptive genetic algorithm for two-stage hybrid flow-shop scheduling with sequence-independent setup time and no-interruption requirement. *Expert Systems with Applications*, 208:118068.

Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.

Shakhlevich, N., Hoogeveen, H., and Pinedo, M. (1998). Minimizing total weighted completion time in a proportionate flow shop. *Journal of Scheduling*, 1(3):157–168.

Sun, R. (2024). Randomized approximation schemes for minimizing the weighted makespan on identical parallel machines. *Journal of Combinatorial Optimization*, 47:34.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Zuo, L., Zhang, J., Shi, Z., and Fan, B. (2026). Single machine scheduling with a restricted rate-modifying activity to minimize the weighted makespan. *Discrete Applied Mathematics*, 380:89–100.